Deployment stack, not model: a reproducibility audit harness for LLM-assisted evidence extraction in environmental assessment

Lucas Rover[a,*], Yara de Souza Tadano[a]

[a] *Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, PR, Brazil*

Abstract

Environmental evidence bases are increasingly assembled with large language models, on the assumption that identical inputs yield identical outputs. We test that assumption with an open audit harness on a screening-and-extraction pipeline over 500 abstracts on fine particulate matter and respiratory health: six deployment stacks, ten runs each (35,636 of 36,000 calls). Local stacks reproduced exactly (Exact Match Rate 1.000); API-served stacks reached 0.150–0.470 on extraction at temperature zero, the most reproducible accepting no seed. The same LLaMA model was deterministic locally but varied across runs when served in the cloud. Neither a fixed-slot prompt nor a decision model returning probabilities removed the variation. Schema conformance was independent of reproducibility (local extraction: 38–43%). A client that logged temperature zero while omitting it shows that provenance must record the transmitted payload. Harness, run cards and human labels (κ=0.529) are released.

Keywords: reproducibility, large language models, deployment stack,

*Corresponding author.

Email address: lucasrover@alunos.utfpr.edu.br (Lucas Rover)

evidence synthesis, quality assurance, provenance, environmental assessment

---

Software and data availability

Name: `llm-repro-harness` — a reproducibility audit harness for LLM-assisted evidence extraction.

Developers: L. Rover and Y. de Souza Tadano, Universidade Tecnológica Federal do Paraná, Curitiba, Brazil. Contact: `lucasrover@alunos.utfpr.edu.br`.

Year first available: 2026.

Hardware required: any POSIX workstation for the audit and analysis layer. Reproducing the local-stack arm additionally requires a machine able to serve 7–9B-parameter models; ours was an Apple M4 with 24 GB unified memory. The API arm requires only network access and provider credentials.

Software required: Python 3.14; Ollama v0.15.5 for the local arm. Dependencies are pinned in `requirements-lock.txt`.

Program language: Python.

Program size: approximately 11,000 lines; the test suite collects 157 cases from 138 test functions.

Availability and cost: free and open source. Code: `https://github.com/Roverlucas/llm-evidence-synthesis-reproducibility` (MIT). Data, run cards and frozen analysis artefacts: OSF project `vr934`, `https://doi.org/10.17605/OSF.IO/VR934` (CC BY 4.0). The human labeling protocol is deposited as an immutable pre-registration, `https://doi.org/10.17605/OSF.IO/FGN3E`.

What the harness does: it runs a task R times against each configured

endpoint through one runner interface (Ollama, Anthropic, Google and OpenAI clients; the same-checkpoint cloud, decision-model and fixed-slot arms use separate scripts), and emits a call record per request (for the six main stacks, a hash of the declared configuration and input; the output hash, timestamps, token counts and, where the runner reads it back from the request body, the temperature actually sent) and a run card per run. From the stored outputs it computes Exact Match Rate with bootstrap intervals, pairwise disagreement, inter-run Fleiss' κ, schema conformance, and—where the task yields effect estimates—the propagation of run-to-run variation into a pooled meta-analytic result. Corpus, prompts and schemas are files; adding a stack means one entry in the model table of `run_experiment.py` and, for a new provider, one runner module. Every result reported here was obtained on one environmental evidence task, $PM_{2.5}$ and respiratory health, and is scoped to it.

1. Introduction

1.1. Evidence Pipelines Are Becoming Software

Environmental evidence bases are increasingly assembled by software rather than by hand. Regulatory reviews, exposure assessments and air-quality standards rest on syntheses of primary literature, and the screening and extraction steps that produce them are now routinely delegated to large language models. Work in this journal has begun mapping that shift, scaling open-weight models for regulatory information extraction in the hydropower domain[1] and examining what structured tool interfaces do and do not provide for LLM-driven model calibration.[2]

That first study also illustrates the assumption this paper sets out to test, and it does so in the way the field as a whole does. Its experimental setup fixes temperature at zero *"thereby eliminating sampling variability and facilitating consistent outputs across multiple experimental runs"*, and executes each model–method permutation once.[1] That is the conventional move: a single execution is treated as sufficient because the decoding parameters are believed to make repetition redundant. The premise is stated openly there, which is what allows it to be tested here; our question is not whether that study was well conducted but whether the premise it shares with the field survives measurement.

Delegating a pipeline step to an LLM makes that step a piece of software, and model and software evaluation is part of this journal's stated scope. Yet the procedure being introduced here carries a property that conventional environmental software does not. A hydrological model run twice on the same inputs returns the same outputs; an LLM endpoint offers no such guarantee, and whether it happens to behave deterministically is a property of how it is served, not of the model named in the methods section. Recent work in this journal has confronted the reproducibility of geoscientific analysis pipelines directly, distinguishing methodological, results and inferential reproducibility, and cataloguing the barriers to each.[3] That catalogue already names the problem we quantify: it lists "non-deterministic model outputs from generative AI" among the barriers to computational reproducibility, and observes that the adoption of large language models introduces reproducibility challenges of a new kind. It does not measure them, and it is not a study designed to. This paper supplies the measurement for one such component, in

the dimension that catalogue calls methodological reproducibility—the exact repetition of a procedure given the same code, data and environment.

Evidence synthesis—screening the literature, extracting effect estimates and pooling them—underpins environmental standards and exposure assessments,[4, 5] and its screening and extraction steps are among the most labour-intensive parts of the work.[6] LLMs now perform those steps with accuracy approaching that of human reviewers,[7–9] and LLM-assisted synthesis is an established methodological line.[10–13] Those evaluations report accuracy from one or two runs; whether a second run returns the same output is what this paper measures. Reporting guidance is beginning to catch up: a 2025 editorial in *Research Synthesis Methods*[14] asks authors of GenAI-assisted syntheses to document random-seed values and, for commercial models, API versions and access dates. What such guidance does not yet establish—and what an audit must supply—is whether documenting those settings buys the determinism their presence implies.

1.2. The Hidden Problem: Non-Determinism

Behind this guidance sits an assumption: *if you ask the same question twice, you should get the same answer.* In the context of LLM-assisted evidence synthesis, this means that running the same model with the same prompt, temperature, and random seed on the same abstract should produce identical screening decisions and extraction outputs every time.

This assumption does not hold in practice.

Atil et al.[15] showed accuracy variations of up to 15% across repeated identical runs, with best-to-worst performance gaps reaching 70%. Klishevich et al.[16] confirmed that temperature=0 does not guarantee determin-

istic outputs across any of the major models tested (GPT-4o, Claude 3.5, LLaMA 3.2). At a deeper level, Fu et al.[17] demonstrated that even when surface-level text appears identical, the underlying token probability distributions exhibit substantial instability, suggesting that apparent determinism may mask hidden variation. A philosophical analysis by Karelin et al.[18] traced this non-determinism to three fundamental sources: numerical precision in floating-point operations, architectural choices in distributed inference, and sampling procedures inherent to the generation process.

1.3. Why This Matters for Evidence Synthesis

In evidence synthesis, run-to-run variation enters the result at three points.

First, decisions propagate. A screening decision that flips from "include" to "exclude" in a repeated run does not merely change one data point—it removes an entire study from downstream analysis. In meta-analysis, where pooled effect estimates are sensitive to study composition, a single inclusion change can shift a confidence interval across the null, transforming a statistically significant finding into a non-significant one.

Second, extraction compounds variation. Data extraction involves free-text parsing of complex scientific results—relative risks, confidence intervals, exposure definitions, population characteristics. Even small variations in how an LLM parses "RR=1.05 (95% CI: 1.01–1.09)" can propagate through meta-analytic pooling. Gartlehner et al.[8] replicated a Claude 2 extraction twice and reported 96.9% and 95.0% accuracy against human extraction; each replication was scored against the reference, not against the other, so the design does not measure how many values change between runs.

Third, a review that cannot be re-executed cannot be audited. The broader reproducibility crisis in AI research[19–21] has already eroded trust in computational findings. When LLM-assisted reviews cannot be independently reproduced because the API returns different outputs, the scientific value of those reviews is undermined.[22–24]

1.4. The Gap We Address

Despite the rapid adoption of LLMs in evidence synthesis, **no study has measured the run-to-run reproducibility of both screening and extraction, and the survival of extraction variation into a pooled estimate**, in an environmental evidence task.

Consider the state of current evidence. Jensen et al.[9] report 94.1% inter-session reproducibility for ChatGPT-4o extraction—an encouraging number. But this was measured with only 2 repetitions and evaluated per-field, not per-output. As we demonstrate below, applying a stricter whole-output metric across 10 repetitions yields extraction EMR as low as 15.0%; because metric, run count, model and corpus also differ from that study, the two figures are not a like-for-like contrast. Similarly, Gartlehner et al.[8] report replication accuracies of 95.0–96.9% against human extraction over two replications, which does not measure how many values change between runs. Studies measuring non-determinism[15, 16] do so on general NLP benchmarks, where the structured, multi-field outputs characteristic of evidence synthesis are absent. And the most influential screening evaluations[7, 12, 25] report results from a single run, implicitly assuming that a repeat would yield the same result.

We address this gap with the following contributions:

1. An open audit harness (`llm-repro-harness`) that runs a task R times against each configured endpoint through one runner interface, emits a per-call record, and computes whole-output Exact Match Rate, pairwise disagreement, inter-run κ and schema conformance;
2. A measurement of run-to-run reproducibility for six deployment stacks over a two-stage environmental evidence pipeline (500 abstracts, 10 runs, 35,636 calls): exact reproduction on the local stacks, extraction EMR of 0.150–0.470 on the API-served stacks at temperature zero;
3. A same-checkpoint comparison (Llama 3 8B Instruct local vs. cloud) showing that a model deterministic on one serving stack varied across runs on another, scoped to the differences in weight precision and request construction between the two arms;
4. Tests of three settings practitioners rely on—a transmitted seed, a fixed-slot output shape, and a decision model that returns probabilities—none of which removed the variation;
5. Evidence that schema conformance and accuracy are separate from reproducibility, and a documented provenance failure in which run cards recorded a temperature the client did not transmit, from which we derive reporting requirements for LLM steps in environmental workflows.

2. Related Work

2.1. LLMs in Systematic Reviews

Oami et al.[7] evaluated GPT-4 Turbo for citation screening, reporting sensitivity of 0.75 and specificity of 0.99, with prompt tuning improving sensitivity to 0.91. Khraisha et al.[25] extended this to multilingual screening and

extraction with GPT-4 and found that, once chance agreement and dataset imbalance were adjusted for, performance was moderate for extraction and ranged from none to moderate for title/abstract screening.

For data extraction, Gartlehner et al.[8] showed that Claude 2 achieved 96.3% accuracy compared to human reviewers, with replication accuracies of 96.9% and 95.0%. Jensen et al.[9] found that ChatGPT-4o achieves 92.4% extraction accuracy and 94.1% inter-session reproducibility, explicitly noting residual stochasticity across sessions.

More comprehensive pipelines have also been proposed. The TrialMind system[26] demonstrated that human-AI collaboration improved recall by 71.4% and reduced screening time by 44.2% across 100 systematic reviews. Li et al.[10] described an LLM pipeline for health technology assessment, while Matsui et al.[12] showed that a three-layer GPT-3.5/GPT-4 strategy achieved human-comparable sensitivity. Cao et al.[27] presented otto-SR, a system that automates the entire systematic review lifecycle with LLMs, demonstrating near-human performance across screening, extraction, and synthesis—yet without examining run-to-run reproducibility. Khan et al.[28] proposed a collaborative multi-LLM approach for automated data extraction in living systematic reviews, achieving high accuracy but reporting results from single executions. A recent systematic review of automated meta-analysis[29] found that 52.5% of 61 studies automate data processing and only one (about 2%) attempts full-process automation—underscoring that evidence synthesis remains a multi-stage pipeline where each step introduces potential variation.

A common thread across these studies is that **reproducibility is either**

assumed or measured only superficially. Test-retest reliability, when reported, is based on 2–3 repetitions and does not examine how variation propagates through multi-stage pipelines.

2.2. The Non-Determinism Problem

Atil et al.[15] provided the most comprehensive analysis to date, documenting accuracy swings of up to 15% across repeated runs and introducing the TARr@N metric to quantify run-to-run stability. Klishevich et al.[16] confirmed that temperature=0 fails to ensure determinism across GPT-4o, Claude 3.5, and LLaMA 3.2 on code review tasks.

At the token level, Fu et al.[17] showed that apparent text-level agreement masks substantial distributional instability in token probabilities—a finding that suggests surface-level reproducibility metrics may underestimate true variation. Angermeir et al.[30] examined how non-determinism undermines reproducibility in software engineering research, proposing documentation standards to mitigate this.

These studies measure non-determinism on general-purpose tasks (benchmarks, code review, classification); none uses the structured, multi-field outputs of evidence extraction.

2.3. Reproducibility in AI Research

The broader AI reproducibility crisis has been documented extensively. Hutson[19] identified unpublished code and sensitivity to training conditions as primary barriers. Haibe-Kains et al.[20] called for structural reform, advocating for mandatory code and model sharing. Kapoor and Narayanan[21] documented data leakage affecting 294 studies across 17 fields, while Han[22]

analyzed reproducibility failures in biomedical AI through the lens of model non-determinism.

Recent frameworks have attempted to systematize reproducibility requirements. Desai et al.[23] proposed a taxonomy of reproducibility types, arguing that LLMs require "conceptual replicability" given their proprietary training data. Semmelrock et al.[24] provided a systematic overview of barriers, and Yu[31] argued that computational reproducibility alone is insufficient for trustworthy AI.

2.4. Provenance and Transparency

Provenance tracking—recording the lineage of data, computations, and decisions—has been identified as a key enabler of AI reproducibility. Longpre et al.[32] audited 1,800+ datasets, finding license omission rates above 70%. Liang et al.[33] analyzed 32,111 model cards on Hugging Face, finding that documentation of limitations and environmental impact remains sparse. The Foundation Model Transparency Index[34] scored 14 major developers across 100 indicators, finding average transparency of only 58/100.

Technical solutions for provenance include the Workflow Run RO-Crate[35] and the W3C Provenance (PROV) Data Model—a W3C Recommendation defining an interoperable standard for representing the entities, activities, and agents involved in producing a piece of data.[36] Longpre et al.[37] argued at ICML 2024 that current practices for data authenticity, consent and provenance are broken, and called for provenance standards.

Our provenance approach builds on the lightweight protocol proposed by Rover et al.,[38] using SHA-256 hashing of inputs, outputs, and configurations to create an auditable trail without requiring infrastructure beyond

standard file storage.

2.5. Reproducibility of Environmental Modelling Workflows

Work in this journal has approached the same problem from the modelling side. Schlögl et al. distinguish methodological, results and inferential reproducibility in geoscientific analysis and list "non-deterministic model outputs from generative AI" among the barriers, without measuring it.[3] Zhu et al. assess the computational reproducibility of geo-simulation experiments workflow-wide and then process by process, a diagnosis that presumes a process behaves the same way when re-executed.[39] On the LLM side, Yoon et al. scale open-weight models for regulatory information extraction and fix temperature at zero to obtain consistent outputs from a single run,[1] and Park et al. show that a typed, range-validated tool interface gives an LLM agent control and reviewable provenance in SWAT+ calibration without guaranteeing a valid model state.[2] None of these measures whether an LLM-mediated step returns the same output when it is run again, which is the per-process property the present harness measures.

2.6. Positioning: What Existing Studies Do Not Address

Table 1 synthesizes the landscape of prior work and highlights the methodological gaps that motivate our study. Three critical gaps emerge. **First**, studies that evaluate LLMs for evidence synthesis[7–9, 12] measure accuracy against human judgments and either do not measure self-consistency across repeated runs or do so with 2–3 repetitions and per-field metrics that can mask whole-output variation (three-run screening consistency of 95.9–98.9% in Oami et al.[13]). **Second**, studies that measure LLM non-determinism[15–

17] do so on general-purpose benchmarks (classification, code review), not on the structured, domain-specific tasks characteristic of evidence synthesis. **Third**, no study in Table 1 measures whether extraction variation survives meta-analytic pooling, or holds the model checkpoint fixed while changing the serving stack inside an evidence-synthesis pipeline. Our study addresses these gaps for one environmental evidence task.

Table 1: Comparison of our study with prior work. "Runs" indicates the number of repeated executions under identical conditions. "Pipe." indicates whether both screening and extraction were evaluated in the same study.

Study	**Domain**	**Task**	**Runs**	**Metric**	**Local**	**Cloud**	**Pipe.**
Oami (2024)	Health	Screening	1	Accuracy	—	✓	—
Oami (2025)	Health	Screening	3	Acc.+consist.	—	✓	—
Khraisha (2024)	Health	Scr.+Extr.	1	Accuracy	—	✓	—
Gartlehner (2024)	Health	Extraction	2–3	Field acc.	—	✓	—
Jensen (2025)	Health	Extraction	2	Field acc.	—	✓	—
Matsui (2024)	Health	Screening	1	Accuracy	—	✓	—
Atil (2024)	General	NLP bench.	10+	Acc. var.	—	✓	—
Klishevich (2025)	Softw.	Code review	5+	Determinism	✓	✓	—
Fu (2026)	General	Token dist.	—	Distrib.	—	✓	—
Li (2025)	Multi	Meta-anal.	—	Survey	—	—	—
Cao (2025)	Health	Full SR	1	Accuracy	—	✓	✓
Khan (2025)	Health	Extraction	1	Field acc.	—	✓	—
This study	**Env. health ($PM_{2.5}$)**	**Scr.+Extr.**	**10**	**EMR+CI**	✓	✓	✓

3. Methods

3.1. Study Design

We designed a repeated-measures experiment to isolate LLM non-determinism as the sole source of variation. We held the corpus, prompts, model configurations, and random seeds constant across all runs. Differences between runs therefore arise from the model or its serving infrastructure, or from calls that failed, which EMR counts as non-matches (Section 3.8.1).

Figure 1 provides an overview of the experimental pipeline.

3.2. Unit of Analysis: Deployment Stacks

We treat the unit of analysis as the *deployment stack*—the tuple (*model_weights*, *provider*, *infras* rather than the model weights alone. For the three local stacks, every component of the tuple is transparent and pinned: open-weight checkpoints with verifiable hashes (LLaMA 3 8B, Mistral 7B, Gemma 2 9B), served by Ollama v0.15.5 on a single Apple M4 chip (24 GB RAM, single replica, all layers on the on-chip GPU, one request at a time), through Ollama's local HTTP endpoint. For the three API-served stacks, only the provider and the client-visible API surface are observable; the underlying weight checkpoint version, GPU topology, batching policy, and intermediate-precision routines are opaque and may change without notice. This framing matters because the local-vs-cloud comparison constitutes a *partial quasi-isolation probe*: when two stacks serve the same checkpoint (Llama 3 8B Instruct, a 4-bit quantization served locally via Ollama and the provider's version via DeepInfra) but differ in infrastructure and serving layer, divergence within each arm is attributable to the non-weight components of the tuple, while divergence be-

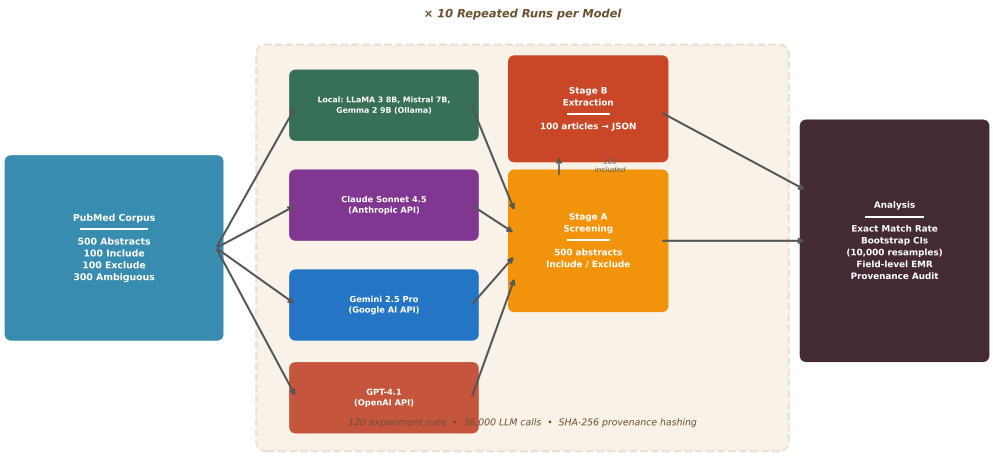


Figure 1: Experimental pipeline. A 500-abstract PubMed corpus on $PM_{2.5}$ and respiratory health is processed by six deployment stacks: three local Ollama/Apple M4 stacks (LLaMA 3 8B, Mistral 7B, Gemma 2 9B) and three API-served stacks (Claude Sonnet 4.5/Anthropic, Gemini 2.5 Pro/Google, GPT-4.1/OpenAI). Each stack is run 10 times under identical configurations (temperature=0, fixed seed where supported). Stage A (screening) produces binary include/exclude decisions over all 500 abstracts; Stage B (extraction) produces structured JSON over the 100 INCLUDE articles. All outputs are SHA-256-hashed for provenance tracking. Reproducibility is quantified via the Exact Match Rate (EMR) with 95% percentile bootstrap confidence intervals.

tween arms also reflects how each request was built and the weight precision each stack serves (Section 4.8). Throughout this paper we use "deployment stack" for the experimental unit and reserve "model" for the weights alone.

3.3. Corpus Construction

We constructed a 500-abstract corpus from PubMed covering the association between $PM_{2.5}$ and respiratory health outcomes (Supplementary Table 26). We chose this domain because it is one of the most extensively studied areas in environmental epidemiology, with Yu et al.[40] estimating approximately 1 million premature deaths annually from short-term $PM_{2.5}$ exposure. Its time-series and case-crossover studies report relative risks or odds ratios with confidence intervals in a small set of formats, which gives the extraction step a defined numeric target. Prior work on particulate matter and respiratory morbidity has shown that machine learning models can capture the complex, lagged relationships between PM_{10} concentration and hospital admissions,[41] further motivating a domain where both the epidemiological patterns and the computational methods are well established. Inclusion criteria required: (1) original epidemiological study, (2) $PM_{2.5}$ as exposure, (3) respiratory hospitalization or emergency-department visit as the outcome—mortality-only studies are *excluded*, (4) time-series or case-crossover design, (5) a relative risk, odds ratio or equivalent effect measure reported with its confidence interval, and (6) publication in English. The screening prompt (`configs/prompts/screening.txt`) and the human labeling protocol state these six criteria; their exact wordings are reproduced in the Supplement. Articles were sourced from PubMed using the E-utilities API with a broad query yielding 573 candidate records, supplemented by

475 exclusion candidates and a design-filtered subset of 222 records. The final corpus spans 148 journals and publication years 1994–2026, with a mean abstract length of 1,873 characters.

Reference labels. The 200 clear-category abstracts (100 include + 100 exclude) were labeled by an automated classification rule based on inclusion-criteria keyword matching; its agreement with human judgement is measured by the dual-human validation below, which contests the rule on its include side. The 300 remaining abstracts received heuristic labels from the same rule operating in low-confidence regions and are explicitly flagged as not human-validated. We do not present these heuristic labels as human labels, and we therefore *exclude* them from the accuracy, sensitivity and specificity computations rather than reporting them separately: those metrics are computed against the 200 clear-category abstracts only, giving per-stack denominators of 192–200 rather than 500 (Table 3). We also record what this stratum is and is not. Of its 300 abstracts, 260 carry the maximum inclusion score of 5—the same score borne by all 100 clear-includes—and all 300 carry zero exclusion reasons. The label "ambiguous" therefore denotes low rule confidence, not measured human disagreement, and the stratum is closer in composition to the include side than the name suggests.

We additionally conducted a **dual-independent human-labeling validation** on a stratified subset of 100 abstracts (25 clear-include + 25 clear-exclude + 50 ambiguous, stratified by heuristic category × year-tertile, seed=42). The labeling protocol, screening templates, and agreement script were pre-registered on the Open Science Framework *before* any label was collected (`https://doi.org/10.17605/OSF.IO/FGN3E`, frozen 2026-05-12; the two la-

bel sets were returned on 2026-07-15 and 2026-07-29). Two reviewers, each blinded to the other's decisions, independently classified every abstract via the protocol in Supplementary §4; we computed Cohen's κ on the 3-class outcome (include/exclude/uncertain) and on the binary include-vs.-exclude collapse. Agreement fell **below the pre-specified target** of $\kappa \geq 0.80$: $\kappa = 0.529$ (SE=0.074, 95% CI [0.383, 0.674]) on the 3-class outcome and $\kappa = 0.556$ (95% CI [0.400, 0.712]) on the binary collapse, with 75% raw agreement and 25 discordant abstracts (confusion matrix and full agreement statistics in Supplementary §4). The shortfall against the target is itself statistically decisive ($z = -3.65$, $p = 1.3 \times 10^{-4}$), whereas $n = 100$ is not sufficient to place κ within a single descriptive band—the interval spans "fair" through "substantial"—so we report the value against the threshold rather than assigning it a label. Two objections can be foreclosed immediately. The low coefficient is not an artefact of skewed marginals: the prevalence index is 0.189 and PABAK (0.558 on the binary collapse) is nearly identical to κ, so the "kappa paradox" is not operating here. Nor is it an artefact of the three-class scale: linearly and quadratically weighted κ (0.541 and 0.548, computed on the ordinal ordering include–uncertain–exclude) move the estimate by less than 0.02. The preregistration specified in advance what to do in this event—report the value transparently, examine the disagreement pattern qualitatively, amend the protocol to address the ambiguity, and file a sub-registration update—and that sequence was followed. The disagreement is not diffuse but directional, and this is testable: the discordant cells of the binary table are 2 versus 19 (exact McNemar $p = 2.2 \times 10^{-4}$), and marginal homogeneity is rejected on the full 3-class table (Stuart–Maxwell $\chi^2(2) = 16.4$, $p = 2.7 \times 10^{-4}$), with a bias in-

dex of 0.179. One rater is systematically more inclusive than the other, which is the signature of divergent protocol reading rather than of random rater noise. The source is specific: 17 of those 19 asymmetric discordances turn on inclusion criterion 5 (a quantitative effect estimate), whose v1.1 wording supported two defensible readings—requiring the numeric estimate and interval to be present, or accepting an explicit statement that the study reported them. The v1.1 decision table compounded this by specifying outcomes for failure in two or more criteria and for a borderline single criterion, but not for clear failure in exactly one, which is precisely the configuration of those 17 abstracts. Neither rater departed from the protocol; the protocol was underspecified. Protocol v1.2 therefore splits criterion 5 into three levels: values present (criterion satisfied), the effect stated but its values absent (*uncertain*, since abstract-only screening cannot verify what the abstract omits), and no estimate or mention at all (*exclude*, since total absence is reasonable evidence that no extractable estimate exists). The amendment also separates structural criteria (original study, $PM_{2.5}$ exposure, respiratory hospitalization, English), where a single clear failure excludes, from conditional ones (design, effect reporting), where a single failure yields *uncertain* unless the information is absent rather than merely unreported. The pre-registered protocol routes the 25 discordant items to a blinded recalibration, a consensus meeting and a tie-break by the senior author; no label from that sequence enters any number reported here, and κ=0.529 is the study's agreement estimate. Showing that the amended criterion repairs the ambiguity requires a fresh independent sample, not a re-rating of the same items; these data do not show it.

Stratifying agreement exposes something the pooled coefficient hides, and it bears directly on how our accuracy numbers should be read. On the 25 clear-*exclude* abstracts the two raters agreed unanimously (25/25, PABAK=1.00; κ is undefined there because expected agreement equals one—a degenerate-marginals case that is precisely why we report PABAK alongside). On the 25 clear-*include* abstracts agreement was 68% (κ=0.359), and on the 50 ambiguous abstracts 66% (κ=0.398). The automated rule's include side fares worst under human scrutiny: of the 25 abstracts it called clearly includable, one rater endorsed inclusion for 13 and the other for 21, and 8 of the 25 discordances fall inside that stratum. The reference is therefore **asymmetrically valid**. Specificity rests on the unanimous exclude side, whose labels are firm but which was drawn by a query for animal, in-vitro and occupational studies, so it measures rejection of clearly off-topic records, not of near-miss studies. Sensitivity rests on include-side labels that the human raters substantially contest, so its error has no known direction: a stack loses credit for correctly rejecting a mislabelled include and gains it for accepting one, and a stack that responds to the same surface features as the rule is favoured. We therefore do not interpret sensitivity differences between stacks.

For extraction, we constructed two independent reference standards: (a) a *silver-internal* consensus from majority vote across all 6 stacks $\times$ 10 runs (60 votes per item), and (b) a *silver-external* consensus from DeepSeek-R1 (5 runs $\times$ 100 items, majority vote per field). DeepSeek-R1 was selected for the silver-external role because it is served by a provider outside those evaluated here; independence of its errors from those of the evaluated stacks

is not established, so the silver-external standard serves as a comparative anchor, not as a reference of truth. Cross-validation between these two silvers yields Spearman ρ=0.845 on `effect_estimate` and 80% agreement across numeric fields (consensus subset, see Supplementary §14), supporting their use as comparative anchors. The extraction set is the 100 abstracts the automated rule labels as includes; of the 25 of them in the human-labelled subset, only 13 are included by first-round human agreement, and 15 abstracts the raters agree to include are outside it. No dual-human extraction reference was built for this study, so extraction accuracy is reported against the two silver standards only.

Because our primary metric (EMR) measures self-consistency rather than accuracy, gold-standard quality affects only the secondary accuracy analysis; we report secondary accuracy against the automated rule's labels (screening) and against the silver-internal and silver-external standards (extraction; the latter on its consensus subset only, see Supplementary §14). Given the asymmetric validity documented above, we treat the exclude side of every accuracy comparison as firm and the include side as uninterpretable.

3.4. Deployment Stacks

We selected six deployment stacks spanning local and API-served paradigms (Table 2). Each stack instantiates the tuple (*model_weights*, *provider*, *infrastructure*, *API_layer*) defined in Section 3.2. The three local stacks cover three open-weight families served on a single device; the three API-served stacks cover three proprietary providers accessed over the public network.

The three local stacks pair open-weight checkpoints—LLaMA 3 8B,[42] Mistral 7B,[43] and Gemma 2 9B[44]—with Ollama v0.15.5 on Apple M4 hard-

Table 2: Deployment-stack configurations. The four-component tuple (*weights*, *provider*, *infrastructure*, *API_layer*) is shown across the first four columns. All local stacks were served via Ollama with deterministic settings. Claude does not expose a seed parameter; Gemini accepts seeds but does not guarantee determinism; GPT-4.1 accepts seeds via the API. Local stacks are 7–9B-parameter open-weight checkpoints; API-served weights are proprietary and approximately >100B parameters, so the local-vs-API comparison varies multiple tuple components simultaneously (see Section 4.8 for a single-component probe).

Weights	**Provider**	**Infrastructure**	**API layer**	**Temp. / Seed**	**top_p**[§]	**Max tokens**
LLaMA 3 8B	Meta (open)	Ollama v0.15.5 / Apple M4 / single replica	local HTTP /api/generate	0 / 42	default	2,048
Mistral 7B	Mistral (open)	Ollama v0.15.5 / Apple M4 / single replica	local HTTP /api/generate	0 / 42	default	2,048
Gemma 2 9B	Google (open)	Ollama v0.15.5 / Apple M4 / single replica	local HTTP /api/generate	0 / 42	default	2,048
Claude Sonnet 4.5 (20250929)	Anthropic	opaque (production)	Messages API v1	0[‡] / —	default	2,048
Gemini 2.5 Pro	Google	opaque (free tier)	generateContent v1beta	0 / 42	default	8,192[†]
GPT-4.1	OpenAI	opaque (production)	Chat Completions v1	0 / 42	default	2,048

[†]Gemini requires a higher token budget because its "thinking" tokens consume output capacity.

[§]Not transmitted; each provider's or Ollama's default applied and was not recorded.

[‡]Run on 8 October 2026, with `temperature`=0 recorded per call from the transmitted payload; the five other stacks ran in February–March 2026. A second execution of the same stack, in which the client omitted the temperature field so that calls ran at the API default of 1.0 (Finding 9), is reported in the Supplement.

ware (24 GB RAM), with identical seed and temperature settings. The LLaMA checkpoint is Ollama's `llama3:8b`, 4-bit quantized (Q4_0). The three API-served stacks accessed proprietary weights through their respective providers: Claude Sonnet 4.5[45] via the Anthropic Messages API (no seed parameter available), Gemini 2.5 Pro[46] via the Google AI `generateContent` API (seed set to 42), and GPT-4.1[47] via the OpenAI Chat Completions API (seed set to 42). All stacks were queried via HTTP (urllib-based, no SDK dependencies) to ensure uniform request handling. Each call was retried up to twice when the request raised an error (exponential backoff) or when the response could not be parsed as JSON (1-second wait); the recorded output is the first parseable response. Attempts were not recorded, so the share of outputs that come from a second or third request is unknown.

A decision-model stack on screening. To test whether returning a distribution instead of a sample removes run-to-run variation, we added a decision-model stack to Stage A only: TypeSafe Jev (`typesafe-ai/jev`), a model that does not generate text but answers a typed question with a choice and the probability of every option. It was called through the Vercel AI Gateway (`POST /v1/evaluate`) with the provider pinned to TypeSafe and no fallback, so every answer was served by the model named here. Each call received, once, the title-and-abstract string that the other stacks receive appended to their prompt, and one `choice` question whose instructions are the inclusion and exclusion criteria of the screening prompt, verbatim; the options were the three labels the generative stacks may return (include, exclude, uncertain). The prompt's JSON-formatting instructions were dropped, because they have no meaning for a model that returns a typed answer; the

article was supplied once rather than twice, and each option carried a one-line definition. These are the differences in the request the stack receives. Jev has no extraction mode, so it does not enter Stage B or the meta-analytic propagation. The arm comprised 10 runs × 500 abstracts, executed on 8 October 2026.

3.5. Sources of Non-Determinism in Distributed Inference

It is useful to distinguish two classes of factors that can plausibly contribute to LLM non-determinism across deployment stacks. *Cloud-deployment factors* are those intrinsic to running a model on hardware accessed over a network: end-to-end network latency, request routing across geographically distributed endpoints, shared physical hardware, and multi-tenancy isolation policies. *Production serving-infrastructure factors* are specific design choices in the LLM inference stack: tensor parallelism with non-deterministic all-reduce, FlashAttention kernel non-determinism, dynamic continuous batching that varies request composition between calls, speculative decoding fast-paths, mixed-precision (BF16/FP16) accumulation, and prefix-caching state shared across requests. These two classes are conceptually distinct: a request that traverses the network can still be served on a deterministic infrastructure, and vice versa, an infrastructure with the non-deterministic features above can be exercised under local-network conditions. Our same-checkpoint comparison (Section 4.8) holds the checkpoint constant while varying the rest of the stack tuple, including weight precision and the request payload, so it does not separate the two classes.

Supplementary Table 27 maps six of these mechanisms onto each of the six deployment stacks studied here, classifying each cell as documented (✓),

likely (consistent with public information from the provider's serving infrastructure), *possible* (compatible with public information but not directly confirmed), or precluded by stack design (—). Attribution is necessarily *inferential* for the API-served stacks because the production serving stacks of OpenAI, Anthropic, and Google are closed-source; the table reflects what is publicly documented or strongly implied by published architecture descriptions, not what we directly measured per request.

The mapping is used in two ways in our discussion. First, the residual disagreement observed in the same-checkpoint cloud experiment (Section 4.8) is compatible with mechanisms 1–4 and with the precision and request differences described there; the design does not discriminate among them. Second, the larger gap observed for the three API-served stacks of much-larger proprietary weights is consistent with all six mechanisms being active simultaneously; we therefore do not attribute it to any single mechanism.

3.6. Prompt Development

Both prompt templates (Listings 1 and 2 in the Supplement) were developed over two pilot rounds before the main experiment.

Round 1. An initial draft was tested on 10 randomly drawn abstracts from the corpus (5 include + 5 exclude) using LLaMA 3 8B locally. Output JSON validated against the schema for 9 of 10 calls; one failure was due to embedded Markdown code-fences. The failure case motivated the explicit instruction "Respond with a JSON object only. Do not include any text outside the JSON." in both prompts.

Round 2. The revised prompt was tested on the same 10 abstracts plus 10 additional abstracts drawn from the ambiguous stratum. Output

validated for 20/20 calls. We additionally manually inspected the rationale text for coherence (no contradictions between rationale and decision); none were identified.

The final prompts (repository commit `75292f2`) were held constant across all 6 stacks and all 120 experimental runs. We did not adapt prompts to individual model capabilities beyond the generic JSON-only constraint, as such adaptation would confound model-level effects with prompt-level optimization. A formal *prompt sensitivity analysis*—varying the structured-output shape (variable-length estimates array vs. fixed-slot single primary estimate)—is reported in Section 4.7.

3.7. Pipeline Stages

Each deployment stack was run 10 times through two stages. Every request carried the title and abstract twice, once inside the filled template and once appended after it; this construction is identical for every stack in Table 2 and in both stages.

Stage A: Abstract Screening. Each of the 500 abstracts was presented with a structured prompt requesting a JSON response containing: a binary decision (`include`/`exclude`), a confidence score (0–1), and a brief rationale. Note that LLaMA consistently processed 480 of 500 abstracts per run (the same 20 abstracts returned output that was not valid JSON in every run); the remaining models processed all 500. Total: 6 stacks $\times$ 10 runs $\times$ 500 abstracts = 30,000 screening calls.

Stage B: Data Extraction. The 100 included abstracts were presented with an extraction prompt requesting: study design, location, period, population, sample size, and a structured list of effect estimates (each with effect

measure, point estimate, confidence interval, lag period, and exposure increment). LLaMA consistently processed 96 of 100 articles per run (the same 4 articles returned output that could not be parsed as JSON); the remaining models processed all 100. Total: 6 stacks $\times$ 10 runs $\times$ 100 articles = 6,000 extraction calls.

In total, the experiment issued **36,000 LLM calls**, of which 35,636 returned an output, across **120 experiment runs**. Stage B receives the same 100 rule-labelled INCLUDE abstracts regardless of any stack's Stage A decisions, so screening variation does not propagate into extraction in this design.

3.8. Reproducibility Metrics

3.8.1. Exact Match Rate (EMR)

Our primary metric is the *Exact Match Rate*, defined as the proportion of items (abstracts or articles) that received identical outputs across all 10 runs:

$$\text{EMR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\,|\{o_{i,1}, o_{i,2}, \ldots, o_{i,R}\}| = 1\,] \tag{1}$$

where N is the number of items submitted (500 for screening, 100 for extraction), a failed call entering as its error output, R=10 is the number of runs, $o_{i,r}$ is the output for item i in run r, and $|\cdot|$ denotes the number of distinct values. For screening, $o_{i,r}$ is the binary decision; for extraction, $o_{i,r}$ is the SHA-256 hash of the full JSON output (used as a compact identity check: any character-level difference produces a different hash).

An EMR of 1.0 indicates perfect reproducibility; an EMR of 0.0 indicates that every item received at least one different output across runs. We note that extraction EMR is a strict metric—any variation in any field (including

formatting differences such as "Beijing, China" vs. "Beijing") produces a non-match. The field-level analysis in Section 4.4 complements this by isolating which fields drive the variation.

3.8.2. Bootstrap Confidence Intervals

We computed 95% confidence intervals for EMR using the percentile bootstrap method with 10,000 resamples. For each bootstrap sample, we drew N items with replacement and computed the EMR, then reported the 2.5th and 97.5th percentiles of the bootstrap distribution. We note that when EMR=1.000 (as for all local stacks), the percentile bootstrap trivially returns [1.000, 1.000]. By the rule of three,[48] observing zero non-matches in N=500 items yields a 95% one-sided upper bound on the true non-match rate of 3/500=0.006; thus, we can rule out population-level non-determinism rates above 0.6% for local models, but not lower rates.

3.8.3. Secondary Metrics

- **Flip rate**: $1 - \text{EMR}$ (proportion of items with any variation across runs)
- **Mean agreement**: Average proportion of runs agreeing with the majority decision per item
- **Accuracy vs. reference labels**: Per-run sensitivity, specificity, and accuracy against the automated rule's labels on the 200 clear-category abstracts
- **Field-level EMR**: For extraction, the proportion of articles with identical values for each individual field across all runs

- **Estimate count stability**: The proportion of articles for which all 10 runs extracted the same number of effect estimates—a measure of structural consistency independent of the specific values extracted

3.9. Provenance Protocol

Scope: client-side observability. The provenance protocol described below records only what is observable from the client. It documents what is observable from the client side—the request payload, the resolved model identifier and version returned by the API (when available), response headers, the client-recorded API response identifier, end-to-end latency, and the input/output text together with their cryptographic hashes. It does not require provider-side weight hashes, GPU-topology fingerprints, or batching-policy attestation, none of which are returned by any production LLM API at the time of writing. The protocol is therefore not restricted to open-source local stacks: for proprietary API-served stacks, it records the client side of each call, the only side a proprietary API exposes. Its operational value is twofold: (i) *differential diagnosis*—when the same prompt produces different outputs at the same client-pinned snapshot, identical call hashes rule out drift in the declared prompt, input and configuration, but not a difference between the declared and the transmitted request (Finding 9); and (ii) *provider-agnostic minimum standard*—adopting the protocol creates a verifiable audit trail across any combination of local and API-served stacks that authors may choose to use. Weight-checkpoint hashes and deterministic-execution attestation would have to come from providers.

Following Rover et al.,[38] we implemented a lightweight provenance protocol that creates a cryptographic fingerprint of every input-output pair,

enabling automated detection of any change across runs:

- **Call-level hashing**: SHA-256 hash of a canonicalized JSON object of the prompt template, input text, model ID, temperature and seed, serialized with sorted keys and ASCII encoding to ensure deterministic hashing across platforms; absent parameters (e.g., Claude's seed) are encoded as `null`. The Claude stack's records add a sixth field naming the hash basis, the same-checkpoint cloud arm was hashed by its own script over a differently formatted input, and the decision-model records carry no call hash
- **Output hashing**: SHA-256 hash of the raw response text
- **Run cards**: Structured JSON metadata per run, including timestamps, token counts, success/failure rates, and aggregate output hashes

The call hash covers the declared configuration, not the transmitted request body. For the Claude stack, each call record additionally stores the temperature read back from the request body as sent (`request_temperature`); the runners of the other stacks place the declared temperature in the request body unconditionally. The protocol is the one proposed in our companion paper,[38] which applies it to nine deployment stacks and six task families with five repetitions per item; the present study applies it to a two-stage pipeline over 500 abstracts at 10 repetitions.

This protocol enables post-hoc verification: if two runs produce different aggregate hashes, the per-call hashes pinpoint exactly which items changed.

3.10. Software and Reproducibility

All code is available at `https://github.com/Roverlucas/llm-evidence-synthesis-reprodu` The experiment was conducted in Python 3.14 on macOS (Apple M4, 24 GB RAM). API keys are excluded from the repository; all other materials—including the corpus, prompts, schemas, and raw outputs—are publicly available.

4. Results

4.1. Overview

Local stacks reproduced every output, in both stages, across ten runs; API-served stacks did not, and the gap widened from screening (EMR 0.936–0.990) to extraction (0.150–0.470). All 120 experiment runs were executed: 60 screening runs (10 per stack × 500 abstracts) and 60 extraction runs (10 per stack × 100 articles), 36,000 calls in all, 35,636 of which returned an output. LLaMA consistently processed 480/500 screening abstracts and 96/100 extraction articles across all 10 runs—the same 20 screening and 4 extraction items failed every time, producing identical error outputs. In screening these failures are not truncation: every such generation ended normally, and inspection shows text that JSON does not admit, such as a comment written after a value; in extraction, one of the four items reached the 2,048-token ceiling in every run. Parsed outputs that violate the schema (for example `case-crossover` where the schema fixes `case_crossover`) are not failures; they are counted as non-conforming in Section 4.10. Because these failures were deterministic (the same items failed identically in every run), they contribute to perfect agreement; EMR was computed over all 500

(screening) and 100 (extraction) items.[1] GPT-4.1 achieved 500/500 screening and 100/100 extraction in all runs, and Claude did the same except for two screening calls in one run that failed on the network and returned no decision; those two abstracts count as non-matches in its EMR. Mistral processed 497/500 screening abstracts and Gemma 493/500, in both cases failing the same items identically in every run, so the footnote above applies to them as well. Gemini achieved 483–500/500 screening and 98–100/100 extraction.

Supplementary Table 28 summarizes the computational resources consumed. The six stacks required 207 hours of inference time and processed 55.8 million tokens. Local stacks were substantially slower per call (mean latency 28–92 seconds) than cloud APIs (3–12 seconds), reflecting the difference between single-device Apple M4 inference and distributed GPU clusters. However, the total cost of local inference was negligible ($0.26 electricity) compared to API costs ($69.38 for Claude and GPT-4.1; Gemini ran on the provider's free tier), with Claude Sonnet 4.5 accounting for 66% of the total budget.

[1]Computing EMR over only the 480 successfully parsed items yields the same result (EMR=1.000), since the 20 consistently-errored items also agree across runs.

4.2. Screening Reproducibility

Important caveat regarding Mistral 7B (Stage A): Although Mistral 7B achieved EMR=1.000 in the screening stage (Table 3), its specificity against the reference labels was 0.240 and its accuracy was 0.628. Inspection of the outputs reveals that Mistral classified 4,660 of its 4,960 parsed screening outputs as "include" (94.0%; 93.2% of the 5,000 attempted)—i.e., a degenerate "always-include" strategy. Any function that returns a constant is trivially reproducible: EMR=1.000 alone is therefore *not* evidence of useful reproducibility. We retain Mistral 7B in our results because this combination (perfect EMR + degenerate specificity) is itself a finding: it warns reviewers that EMR is necessary but not sufficient to characterize a screening tool's utility. We discuss the implications for paired-metric reporting in Section 5.1 of the Discussion.

Table 3 presents the screening reproducibility results.

All three local stacks—LLaMA, Mistral, and Gemma—produced identical screening decisions across all 10 runs for every abstract (EMR=1.000), so no run-to-run variation was observed (non-match rate $\leq$0.6%, rule of three). Among the cloud APIs, Claude showed a 1.0% flip rate (5 abstracts, two of them the network failures noted above; as for every stack, a failed call counts as a non-match, and without those two the rate is 0.6%), GPT-4.1 3.0% (15 abstracts), and Gemini 6.4% (32 abstracts, 17 of them involving a failed call; 15 in which every run returned a decision).

Notably, accuracy and reproducibility behaved as independent properties: Gemini achieved the highest accuracy (0.964) and a reproducibility (EMR=0.936) below Claude's (paired McNemar, Holm-adjusted p<0.001; Supplementary Table 18) and not distinguishable from GPT-4.1's, while

Table 3: Screening reproducibility across 10 runs per deployment stack. LLaMA's 20 consistently-errored abstracts are included in the EMR denominator (see text). For stacks with EMR=1.000, we report the rule-of-three 95% upper bound on the non-match rate ($3/n$[48]) instead of the trivial bootstrap interval. Accuracy, sensitivity and specificity are computed against the automated rule's labels on the 200 clear-category abstracts (denominators 192–200).

Deployment Stack	**EMR**	**95% UCL***	**Flip Rate**	**Accuracy**	**Sensitivity**	**Specificity**
LLaMA 3 8B / Ollama / M4	1.000	≤0.006	0.0%	0.927	0.928	0.926
Mistral 7B / Ollama / M4	1.000	≤0.006	0.0%	**0.628**	**1.000**	**0.240**‡
Gemma 2 9B / Ollama / M4	1.000	≤0.006	0.0%	0.909	0.969	0.848
Claude Sonnet 4.5 / Anthropic	0.990	[0.980, 0.998]	1.0%	0.935	0.868	1.000
Gemini 2.5 Pro / Google	0.936	[0.914, 0.956]	6.4%	0.964	0.929	1.000
GPT-4.1 / OpenAI	0.970	[0.954, 0.984]	3.0%	0.941	0.882	1.000

*For EMR=1.000: 95% rule-of-three upper bound on non-match rate, $3/n$=3/500=0.006. For EMR<1.000: percentile bootstrap CI.

‡Mistral 7B exhibits a degenerate "always-include" strategy (sens=1.000, spec=0.240). EMR=1.000 is trivial here; see warning box.

Mistral had perfect reproducibility but the lowest accuracy (0.628) due to an include-biased strategy (sensitivity=1.000, specificity=0.240). This illustrates that a stack can be accurate on average yet inconsistent across individual runs, and conversely, that perfect reproducibility does not imply high accuracy.

Inter-run agreement (Fleiss' κ). As a chance-corrected complement to EMR, we computed Fleiss' κ across the 10 runs on the 3-class screening decision (include/exclude/uncertain) for each deployment stack. Inter-run Fleiss' κ for screening ranged from 0.974 (GPT-4.1) to 1.000 (all three local stacks); the cloud APIs were κ=0.995 (Claude), 0.975 (Gemini), and 0.974 (GPT-4.1). These values quantify self-consistency across repetitions of the same stack and are not directly comparable to the $\kappa \geq 0.80$ target we prespecified for agreement between two human reviewers, which concerns a different construct (see Section 5.1). For whole-output extraction (each unique output hash as a distinct rating category), the picture is dramatically different: Claude's inter-run κ=0.757, Gemini's κ=0.505 and GPT-4.1's κ=0.512; local stacks retain κ=1.000. Because each output hash is its own category, chance agreement is near zero and these values approximate one minus the pairwise disagreement of Table 4; we attach no descriptive band to them. Full Fleiss' κ values with 95% CIs and per-categorical-field decompositions are reported in Supplementary Table 19.

4.3. Extraction Reproducibility

The extraction results reveal a dramatically larger reproducibility gap (Table 4).

Table 4: Extraction reproducibility across 10 runs per deployment stack. The "Pairwise" column reports the mean fraction of pairs (across $\binom{10}{2}$=45 run-pairs per article) producing different outputs—a more powerful indicator than EMR alone of how dispersed flipping items are. For EMR=1.000 cells we report the rule-of-three upper bound (3/100=0.030).

Deployment Stack	**EMR**	**95% UCL***	**Pairwise**	**Estimate Stability**
LLaMA 3 8B / Ollama / M4	1.000	≤0.030	0.0000	1.000
Mistral 7B / Ollama / M4	1.000	≤0.030	0.0000	1.000
Gemma 2 9B / Ollama / M4	1.000	≤0.030	0.0000	1.000
Claude Sonnet 4.5 / Anthropic	0.470	[0.370, 0.570]	0.241	0.978
Gemini 2.5 Pro / Google	0.200	[0.120, 0.280]	**0.493**	0.735
GPT-4.1 / OpenAI	0.150	[0.080, 0.220]	0.486	0.922

*95% rule-of-three upper bound on non-match rate when EMR=1.000.

While screening EMR remained above 0.93 for all stacks, extraction EMR dropped for all three cloud APIs: **0.150 for GPT-4.1** (only 15 of 100 articles produced identical extraction outputs across all 10 runs), **0.200 for Gemini** (20 of 100), and **0.470 for Claude** (47 of 100). All three local stacks again achieved perfect extraction reproducibility (EMR=1.000). Claude showed the highest estimate count stability (0.978), followed by GPT-4.1 (0.922) and Gemini (0.735): for Claude and GPT-4.1, the outputs varied while the structural composition of the extractions stayed largely fixed.

We emphasize that extraction EMR is computed over the full JSON output hash, so any character-level variation counts as a mismatch. To assess practical significance, we examine field-level variation below.

4.4. Field-Level Analysis

Not all extraction fields are equally affected (Supplementary Table 29). A clear pattern emerges: **categorical fields** (study design, study period) are highly reproducible even for API-served stacks (EMR≥0.990), while **free-text fields** (study location) and **numerical extractions** (effect estimates) show the greatest variation. GPT-4.1 is notable for achieving perfect location reproducibility (1.000) while showing variation in categorical fields like study design and period (0.990 each)—a different instability profile from Claude and Gemini, where location is the most variable field.

The estimate count stability metric reveals that 26.5% of Gemini's articles with extractable estimates, 7.8% of GPT-4.1's, and 2.2% of Claude's received a different number of effect estimates across runs.[2] This structural variation directly affects any downstream meta-analysis.

4.5. Semantic Equivalence Analysis

The hash-based EMR reported in Table 4 (0.150 for GPT-4.1, 0.200 for Gemini, 0.470 for Claude) treats any character-level difference in the *full JSON output*—including all numeric effect estimates, confidence intervals, and variable-length estimate arrays—as a mismatch. To disentangle variation in the five metadata fields from variation in numeric estimates, and to assess whether metadata variation is merely cosmetic (formatting, abbreviation) or

[2]Estimate count stability is computed over articles that produced at least one estimate in all 10 runs, yielding denominators of 83 (Gemini), 90 (GPT-4.1), and 89 (Claude) rather than the full 100. The meta-analytic "Varying" column in Supplementary Table 20 uses the full 100-article denominator, producing different percentages (20%, 5%, 1%), since both the numerator and the denominator change.

genuinely semantic, we applied three progressively relaxed string criteria and one embedding-based criterion to the metadata fields only (Supplementary Table 30).

1. **Exact match**: character-identical outputs (equivalent to SHA-256 hash comparison).
2. **Normalized match**: after lowercasing, whitespace normalization, punctuation stripping, and abbreviation expansion (e.g., "U.S." → "United States").
3. **Fuzzy match**: Levenshtein similarity ratio ≥ 0.90 between all normalized pairwise comparisons across runs.
4. **BERTScore**: Embedding-based semantic similarity using `roberta-large`, computing the F1 score across all $\binom{10}{2}=45$ unique run pairs per article (4,500 pairs per stack).

Four findings emerge from this multi-level analysis.

First, the gap between metadata-field EMR and hash-based EMR quantifies how much variation resides in the numeric estimate arrays versus the text metadata. For Claude, 93% of articles had identical metadata across all runs, but only 47% had identical *complete* outputs; for GPT-4.1, 88% had identical metadata but only 15% had identical complete outputs; for Gemini, 62% and 20% respectively. In all three cases, the variable-length estimate arrays (effect sizes, confidence intervals) are the primary source of non-reproducibility.

Second, within the metadata fields, normalization and fuzzy matching improved EMR by at most 1 percentage point (Gemini: 0.620 → 0.630;

Claude: 0.930 and GPT-4.1: 0.880 unchanged), showing that the variation is not a formatting artefact: the values themselves differ.

Third, BERTScore embedding similarity computed across all $\binom{10}{2}$=45 unique run pairs per article (4,500 pairs per stack) was near-perfect for all API-served stacks: GPT-4.1 averaged F_1=1.000 ± 0.004, Claude F_1=0.999 ± 0.004, and Gemini F_1=0.997 ± 0.022, with 100%, 100%, and 98.5% of pairs exceeding $F_1 \geq 0.95$, respectively. These means include the identical pairs, which dominate, so they say little about the pairs that differ. This creates an apparent paradox: the models produce *semantically near-identical* metadata text, yet 53–85% of their complete extraction outputs are non-identical. The resolution lies in the numeric estimates—the effect sizes, confidence intervals, and variable-length estimate arrays that hold hash-based EMR at 0.150–0.470. Even when a model describes a study in nearly the same words, it may extract a different number of effect estimates or parse "RR=1.05 (95% CI: 1.01–1.09)" with subtly different numeric precision. Concrete, verbatim side-by-side excerpts of an exact-match local-stack output and a near-miss API-stack output (with the divergence located in the variable-length `estimates` array rather than in the metadata fields) are reproduced as JSON listings in Supplementary §5.

Fourth, the three API-served stacks exhibit distinct instability profiles. Claude shows the highest metadata stability (exact EMR=0.930) and the highest hash-based EMR (0.470), yet more than half of its complete outputs still differ across runs, which places its residual variation in the numeric layer. GPT-4.1 shows high metadata stability (0.880) and the highest BERTScore (F_1=1.000), yet its hash-based EMR (0.150) is below Claude's and not dis-

tinguishable from Gemini's (Supplementary Table 18), so numeric extraction drives its variation even more strongly. Gemini shows the lowest minimum BERTScore (F_1=0.754) and highest standard deviation (0.022), indicating that some articles receive genuinely divergent interpretations across runs—not merely numeric rounding differences.

4.6. Downstream Propagation

An audit of an extraction step is only as interesting as what the step feeds. We therefore propagated each run's extracted estimates into a pooled analysis, to ask whether run-to-run variation survives aggregation or is averaged away.

The pooled quantity is a propagation instrument, not an estimate of the $PM_{2.5}$ effect: estimates were pooled on the log scale as extracted, without harmonizing exposure increment, lag, outcome or effect measure. Over the full corpus the run-to-run variation is largely averaged away: no run moves the pooled 95% interval across RR=1. The variation surfaces at small k. Resampling subsets of ten studies, the share of subsets whose conclusion flips depending on which run supplied the inputs reaches at most 2.0% under DerSimonian–Laird and as much as 45.0% under the Hartung–Knapp–Sidik–Jonkman correction, both on the Gemini stack; for GPT-4.1 the HKSJ figure is 21.5%, and for Claude and the three local stacks no subset flips. A second, quieter effect is that k itself moves with the run—on the Gemini stack between 39 and 45 articles yield a usable estimate depending on the execution—so non-determinism changes not only the numbers entering the pool but how many there are.

Because the pooling machinery is peripheral to the audit itself, the full

analysis, the per-run inputs and the heterogeneity statistics are reported in Supplementary §19.

4.7. Fixed-Slot Extraction Sensitivity

To test whether the variable-length `estimates` array structurally amplifies cloud API non-determinism, we ran the extraction stage on the three API-served stacks with a fixed-slot prompt that requests *exactly one* primary estimate (lag 0–1 if multiple lags are reported, single-pollutant model preferred, all-respiratory preferred over subgroups, normalized to per 10 μg/m^3). Temperature (0), seed (42 where the API accepts one) and the 100 INCLUDE articles were those of the variable-length design, with three runs per stack. Three settings differ: the Gemini and GPT-4.1 fixed-slot runs were executed on 26 April 2026, two months after their baselines (the Claude runs on the same day as its baseline); the output-token ceiling was 1,500 (Gemini 4,096) instead of 2,048 (8,192); and output identity in this arm is the SHA-256 of the parsed JSON re-serialized with sorted keys. Table 5 therefore hashes the baseline outputs the same way.

The baseline must be matched on run count. EMR is monotonically non-increasing in the number of runs, since each additional run is another opportunity for an item to differ. The fixed-slot condition was executed with 3 runs while the variable-length baseline has 10, so comparing them directly credits the fixed-slot prompt with a difference that is partly an artefact of fewer repetitions. We therefore compute the baseline over all $\binom{10}{3}$=120 three-run subsets and average, which is what Table 5 reports. With the baseline hashed on the raw response text instead, the relative changes are −3.0%, −47.9% and +23.8% (Supplementary §12).

Table 5: Fixed-slot extraction EMR against a run-count-matched variable-length baseline. Both columns are 3-run EMR with output identity defined on the parsed JSON: the fixed-slot condition as executed (Wilson 95% interval in brackets), and the baseline averaged over all $\binom{10}{3}$=120 three-run subsets of the 10 variable-length runs. Coverage differs slightly by stack—the fixed-slot arm returned usable output for 100 articles on Claude and 99 on Gemini and GPT-4.1—and each EMR is computed over the articles present in all runs of its own arm.

Deployment Stack	**Var-length, 3-run**	**Fixed-slot, 3-run**	**Relative change**
Claude Sonnet 4.5 / Anthropic	0.678	0.640 [0.542, 0.727]	−5.6%
Gemini 2.5 Pro / Google	0.406	0.182 [0.118, 0.269]	−55.2%
GPT-4.1 / OpenAI	0.361	0.444 [0.350, 0.543]	+23.1%

Fixed-slot EMR was 55% lower than the matched baseline on Gemini, outside the fixed-slot interval; on GPT-4.1 (+23%) and Claude (−6%) the baseline lies inside it. These data come from one three-run execution per stack and do not show that constraining the output shape reduces variation. The result converges with one obtained independently in this journal, on a different environmental task and from the opposite starting point. Constraining an LLM agent to operate SWAT+ exclusively through a typed, range-validated tool interface shortened its calibration path substantially (21.9 versus 62.0 model evaluations) and produced reviewable operational provenance—so the structure delivered on control. What it did not deliver was validity: an acceptable goodness-of-fit score did not guarantee hydrological quality, and a held-out corruption task showed that logs and range constraints alone could not detect a physically corrupted model state.[2] Park et al. measure validity and we measure reproducibility; in both studies a structural constraint on

the LLM's interface did not by itself secure the property measured.

4.8. Same-Checkpoint Comparison Across Serving Stacks

Throughout the comparisons above, the local versus cloud difference is confounded with model size: our three local models are 7–9B-parameter open-weight models, while the three cloud APIs are estimated at >100B parameters and trained with proprietary procedures. To probe the deployment effect, we ran an additional condition: the same Llama 3 8B Instruct checkpoint served via cloud (`meta-llama/llama-3-8b-instruct`, OpenRouter routing pinned to the DeepInfra provider) at temperature=0 and seed=42. The local stack serves Ollama's 4-bit (Q4_0) quantization of that checkpoint; the provider does not disclose the precision it serves. Ten cloud runs × 500 abstracts mirror the local experimental design.

The two arms hold constant the checkpoint, the prompt source file, the decoding temperature and the seed; they differ in weight precision and in the request payload. The harness client used for every main stack fills the template's title and abstract slots and then appends the title and abstract a second time; the cloud script sends a single user message containing the template with its `{title}` and `{abstract}` placeholders left unfilled, followed by the article once. The two were also configured with different output-token ceilings (2,048 local, 512 cloud). Measured over the deposited call records, the arms differ by 34.2% in mean input tokens (1,279.3 local vs. 842.3 cloud). The comparison therefore shows how each arm behaves across its own repeated runs; it does not isolate the serving infrastructure from weight precision or from payload construction, and it covers screening only. Output truncation is excluded as a contributor by direct measurement: all 4,266 completed cloud

calls returned `finish_reason=stop`, none hitting the 512-token ceiling.

Table 6: Same-checkpoint, different-stack comparison. Two deployment stacks share the Llama 3 8B Instruct checkpoint (4-bit locally, undisclosed precision in the cloud) and differ in the remaining tuple components: local stack = (weights, Meta/open, Ollama v0.15.5/Apple M4/single replica, local HTTP); cloud stack = (weights, Meta-via-DeepInfra, OpenRouter routing/opaque GPU pool, OpenRouter Chat Completions). Ten runs each, same prompt source file, seed=42, temperature=0; the request payloads are not byte-identical (see text).

Quantity	**Local stack (Ollama / M4)**	**Cloud stack (DeepInf**
1 − mean pairwise disagreement	1.000	0.992*
Mean pairwise disagreement	0.0000	0.0079
% items with any pair-disagreement	0.00%	2.04%
Cross-deployment agreement (all run pairs)	—	59.7%
Items where local=include / cloud=exclude	—	167
Items where local=exclude / cloud=include	—	**0**

*Not an EMR: no abstract was validly decided in all 10 cloud runs, so EMR (Eq. 1) is undefined; pairs with a failed call are excluded. 2 of 10 cloud runs had transient infrastructure failures that reduced their valid-item counts (45 and 221, vs. ∼491 in the other 8 runs).

Three findings stand out.

First, a small-parameter model became non-deterministic when served through a different stack. Local LLaMA 3 8B is bit-deterministic (EMR=1.000 over 10 runs and 500 abstracts). The same model served via cloud is *not*: 2.04% of items receive different decisions in at least one run-pair (mean pairwise disagreement 0.79%). Small model size therefore does not by itself ensure determinism. Because the two stacks also differ in weight

precision and request construction, the result does not separate serving infrastructure from either.

Second, the two arms reach different decisions. Even within the runs where the cloud was internally consistent, mean pairwise agreement with the local deployment was 59.7%. Comparing the canonical run of each arm, of 430 items where both deployments produced a valid decision, 167 (38.8%) showed a different decision under cloud than under local.

Third, the discordance is asymmetric. Of those 167 discordant items, **all 167** were cases where the local model said *include* but the cloud said *exclude*; the reverse direction (local exclude / cloud include) occurred zero times. The direction is systematic, and it cannot be assigned to the serving stack: the two arms rendered the request differently (the cloud prompt showed the template slots unfilled) and served different weight precisions, so serving-side numerics, quantization and payload construction are all candidates and this design does not discriminate among them. The within-arm result is unaffected, because each arm repeated its own payload unchanged.

We additionally observed an operational reliability finding: 2 of 10 cloud runs (`run_005`, `run_006`) experienced transient infrastructure failures that reduced their successful-call counts substantially (45/500 and 221/500 versus the ∼491/500 baseline of the other eight runs). This pattern was absent from all 10 local Ollama runs, all of which had identical 480/500 success. Cloud reliability is therefore a second property to report alongside reproducibility.

4.9. A Decision Model on the Screening Stage

The generative stacks sample one answer per call. A decision model returns the whole distribution, and a natural expectation is that a stack

which never samples should never disagree with itself. The Jev arm tests that expectation on Stage A.

It completed 4,998 of 5,000 calls; the two failures were a server error and a dropped connection. Screening EMR was 0.956 (95% CI [0.938, 0.972]): 20 abstracts received a different decision in at least one run and the two failed calls add two non-matches, within the range of the generative API stacks (0.936–0.990). Against the same reference labels as Table 3, accuracy was 0.888, specificity 1.000 and sensitivity 0.776, the lowest sensitivity of any stack in Table 3.

The probabilities show where the variation comes from. The probability of the chosen option varied across the ten runs by 0.039 on average, reached 0.20, and was zero for only 29.8% of abstracts. Per option the picture is the same. For the same abstract and the same question, the probability Jev assigned to *include* was identical across all ten runs for only 31.4% of abstracts; its range across runs averaged 0.037 and reached 0.19, and over all three options the largest range was 0.20. Most of this movement does not change the decision, because the modal option carried a mean probability of 0.865 and fell below 0.5 for 4.4% of abstracts. Returning the distribution therefore does not remove run-to-run variation. Abstracts whose probabilities move between runs can be identified from the output; whether routing them to a human reader improves screening was not tested. We make no calibration claim from these data: the human-labelled subset (n=100) is too small to estimate one. Call-level details are in Supplementary §24.

4.10. Schema Conformance: A Second Axis of Reliability

EMR asks whether repeated runs agree with one another. It is silent on whether what they agree upon is usable. These are separable properties, and separating them changes how the local stacks should be read.

Table 7: Share of returned outputs validating against the study's JSON schema, over all runs. Calls that returned nothing are excluded from the denominator and counted separately. Conformance is independent of EMR: a stack can reproduce a non-conforming output perfectly.

Deployment Stack	**Screening**	**Extraction**
LLaMA 3 8B / Ollama / M4	87.0%	38.0%
Mistral 7B / Ollama / M4	84.2%	43.0%
Gemma 2 9B / Ollama / M4	96.2%	40.0%
Claude Sonnet 4.5 / Anthropic	100.0%	72.4%
Gemini 2.5 Pro / Google	99.7%	**2.6%**
GPT-4.1 / OpenAI	100.0%	64.9%

Two readings follow, and they cut in opposite directions.

The first tempers the local-stack result. The three local stacks reach EMR=1.000 on extraction while returning schema-conforming output in only 38–43% of calls. Their perfect reproducibility is therefore reproducibility of a set that includes a majority of records a review team would have to repair by hand. This does not overturn the reproducibility finding—an unusable output that is identical across runs is still identical—but it does mean that "deterministic" and "ready to use" are not the same claim, and this paper only establishes the first.

The second sharpens the cloud result. Gemini validates on 2.6% of extraction calls, 951 of its failures being a null in a field the schema types as a string. The meta-analytic propagation in Section 4.6 nonetheless pools 154–171 estimates from 39–45 articles per Gemini run, drawn from outputs the schema validator rejects in 97 of every 100 cases. A pipeline reporting only pooled effect sizes would surface none of this.

Conformance is thus a reliability axis orthogonal to EMR, cheap to measure, and absent from the studies in Table 1. We report it because the pairing is what carries the lesson: Mistral shows that perfect EMR can accompany a degenerate decision rule, and the local extraction stacks show that it can accompany a largely non-conforming output. Neither is visible if reproducibility is measured alone.

4.11. Summary Comparison

Figure 2 visualizes the EMR comparison across stacks and stages, and Figure 3 displays the field-level variation as a heatmap.

The results reveal three key patterns: (1) a sharp reproducibility gap between local and API-served deployment stacks, with all three local stacks achieving EMR=1.000 while all three API-served stacks reach at most 0.470 for extraction; (2) whole-output extraction changing far more often than the screening decision; and (3) distinct instability profiles among API-served stacks, with the Claude / Anthropic stack showing the highest metadata and structural stability but still returning a different complete output in more than half of the articles.

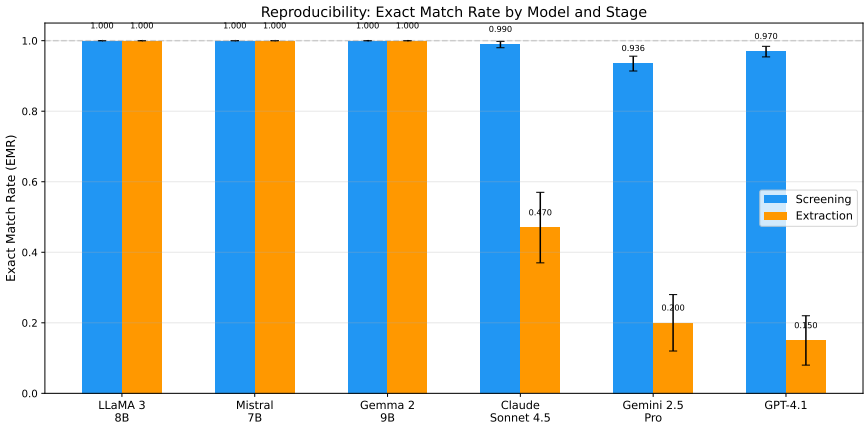


Figure 2: Exact Match Rate (EMR) by deployment stack and pipeline stage. Six deployment stacks are shown: three local Ollama/Apple M4 stacks (LLaMA 3 8B, Mistral 7B, Gemma 2 9B) and three API-served stacks (Claude Sonnet 4.5/Anthropic, Gemini 2.5 Pro/Google, GPT-4.1/OpenAI). Error bars represent 95% percentile bootstrap confidence intervals over 10,000 resamples; for stacks with EMR=1.000, the displayed upper bound is the rule-of-three $3/n$ instead of the trivial bootstrap interval. The dashed line indicates perfect reproducibility (EMR=1.0).

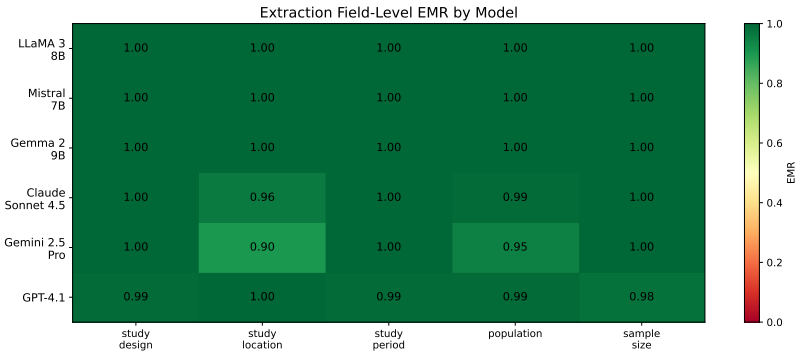


Figure 3: Field-level EMR heatmap for extraction outputs across the six deployment stacks (three local Ollama/Apple M4 stacks on the left columns; three API-served stacks—Claude/Anthropic, Gemini/Google, GPT-4.1/OpenAI—on the right). Darker cells indicate lower reproducibility. Categorical fields (study design, study period) remain stable across all stacks, while free-text fields (study location) and the variable-length structured estimates array show the greatest variation in the three API-served stacks.

5. Discussion

5.1. Principal Findings

Among the studies in Table 1, this is the only one combining whole-output metrics, ten repetitions and propagation of extraction variation into pooled estimates.

Reframing: the object of reproducibility variation is the deployment stack, not the model. A first-principles consequence of our results is that "which model" is the wrong level of abstraction for talking about LLM reproducibility. A deployment stack—the tuple of weights, provider, infrastructure, and API layer (Section 3.2)—is the unit through which an LLM actually generates an answer, and it is the unit at which reproducibility is preserved or destroyed. Two stacks that share a checkpoint need not share

determinism: the same 8B model reproduced exactly on the local stack and varied on the cloud stack (Table 6); two stacks that differ in weights but share a deterministic local infrastructure can both achieve EMR=1.000 (Tables 3, 4). Reporting only the model name in an LLM-assisted SR therefore underspecifies the experimental condition: the same checkpoint on a different stack did not reproduce the same outputs here. This reframing motivates the recommendations in Section 5.5.

Finding 1: Determinism belonged to the local single-device stacks, and a small model lost it when served in the cloud. All three local stacks—LLaMA 3 8B, Mistral 7B, and Gemma 2 9B served via Ollama on a single device—produced identical outputs across all 10 runs in both screening and extraction. All three cloud-based APIs (Claude, Gemini, GPT-4.1) exhibited measurable non-determinism despite transmitting temperature=0 and, where the API accepts one, a fixed seed. The same-checkpoint comparison (Section 4.8) shows that small model size does not confer determinism: the 8B model that reproduced exactly on the local stack left 2.04% of abstracts with a discordant run pair (mean pairwise disagreement 0.79%) when served through DeepInfra. It covers screening only and differs between arms in weight precision and request construction, so it does not show that size plays no part in the extraction gap. The gap is consistent with floating-point non-associativity under multi-GPU serving,[18] where partial sums are reduced in an order that can change between calls, whereas a single device serving one request at a time can repeat the same order; the design does not isolate that mechanism. An INT4-quantized serving of the same model through a different cloud provider reproduced exactly

on a 10-abstract extraction task in our companion study,[38] so the variation observed here belongs to this serving stack, not to cloud serving as such.

Finding 2: The whole extraction output varies far more often than the screening decision, and the variation is in content, not formatting. Claude's EMR dropped from 0.990 (screening) to 0.470 (extraction); GPT-4.1's from 0.970 to 0.150; Gemini's from 0.936 to 0.200. Screening requires a binary decision (a single token effectively determines the output), while extraction generates complex structured JSON with multiple fields. The two EMRs are computed on different outputs—the decision alone versus the full JSON—so the drop does not measure how much the task itself amplifies non-determinism. The semantic analysis (Section 4.5) shows that the extraction gap is not an artifact of strict string matching: metadata text is near-identical across runs (BERTScore $F_1 \geq 0.997$, a mean dominated by identical pairs), yet hash-based EMR remained at 0.150–0.470. This paradox reveals that the variation concentrates in the numeric extraction layer: the models describe studies in nearly the same words but extract different numbers of effect estimates and parse numeric values with different precision. Even the residual metadata variation is in content (fuzzy matching recovers at most 1 percentage point), not formatting. The practical implication depends on context: the high BERTScore ($F_1 \geq 0.997$) indicates that API-served stacks describe studies in near-identical terms, and for applications requiring only qualitative summaries, this level of consistency may be acceptable. However, for quantitative evidence synthesis—where exact numeric values feed directly into meta-analytic pooling—the hash-based EMR of 0.15–0.47 represents a genuine threat to reproducibility.

Finding 3: API-served stacks exhibit distinct instability profiles. Claude achieved the highest metadata stability among API-served stacks (exact EMR=0.930), the smallest structural variation (Δk=2 estimates across runs) and the highest hash-based extraction EMR (0.470), yet more than half of its complete outputs still differed across runs. GPT-4.1 paired high metadata stability (0.880, BERTScore F_1=1.000) and a small structural swing (Δk=4) with a hash-based EMR of 0.150, while Gemini showed the widest structural swing (Δk=17) and the most extreme semantic divergence (minimum BERTScore F_1=0.754). These distinct profiles suggest that non-determinism manifests differently across API providers—some vary primarily in numeric precision, others in structural composition, and others in semantic interpretation.

Finding 4: Accuracy and reproducibility are independent properties; report them as a paired tuple. Gemini achieved the highest screening accuracy (0.964) and a screening reproducibility (0.936) below Claude's, while Mistral had perfect reproducibility (EMR=1.000) but a degenerate "always-include" strategy (specificity=0.240, accuracy=0.628). This independence means that **accuracy evaluations from a single run cannot detect reproducibility problems**, and **reproducibility evaluations without accuracy cannot detect degenerate strategies**. EMR and accuracy should be reported as a paired tuple, never EMR alone.

Finding 5: Constraining the output shape did not remove the variation. In the fixed-slot test (Section 4.7), read against a run-count- and identity-matched baseline, fixed-slot EMR was lower on Gemini (−55%, outside the fixed-slot interval) and not distinguishably different on GPT-4.1

(+23%) and Claude (−6%). The test is one three-run execution per stack, two months after the baseline for Gemini and GPT-4.1, so it shows only that the output shape did not account for the variation here.

Finding 6: Neither a transmitted seed nor temperature zero delivers determinism. Recent reporting guidance for GenAI-assisted evidence syntheses asks authors to document random-seed values and, for commercial models, API versions and access dates;[14] it does not establish whether a documented seed delivers determinism.

The direct evidence needs no cross-provider comparison. Gemini and GPT-4.1 both expose a seed parameter, both received seed=42 on every one of their 6,000 calls, and both clients place `temperature`=0 in every request body (verified in the client code). Under those conditions—the exact configuration a practitioner would adopt believing it buys reproducibility—their extraction EMR was 0.200 and 0.150 respectively, against 1.000 for local stacks holding the same nominal settings. A documented, accepted, transmitted seed therefore does not make an API-served extraction pipeline reproducible.

The data also contain an observation that runs against the seed's reputation, and we state it with its limit. The most reproducible API-served stack on extraction was the one that accepts no seed at all: Claude, at temperature=0 and without a seed, reached 0.470 against 0.200 and 0.150 for the two seeded stacks. That is a contrast between stacks that differ simultaneously in provider, infrastructure and API layer, so it is not an estimate of the seed's effect—our own unit-of-analysis argument in Section 3.2 forbids that reading—but it is incompatible with the seed being what separates

reproducible from non-reproducible API stacks. What our design does *not* support is an estimate of how much the seed contributes on its own. Such an estimate requires a within-stack ablation—the same stack, same provider, same infrastructure, run with and without the seed—and we did not execute one.

Temperature behaves the same way. The Claude execution in which the client omitted the temperature field, so that calls ran at the API default of 1.0 (Finding 9; Supplementary §23), gives a contrast: its extraction EMR was 0.050 at 1.0 (February) and 0.470 at 0 (October), its screening EMR 0.974 and 0.990, yet more than half of the articles still received a different complete extraction across runs. The two executions are eight months apart (Section 5.4), so the contrast combines the temperature change with any provider-side change in between. Temperature zero does not remove the variation, and the lever is closing: the provider's documentation states that its newer models return an error when `temperature` is set to a non-default value,[3] so on those models a practitioner can no longer request 0 at all.

Finding 7: A model that returns probabilities instead of a sample still varies. The decision model (Section 4.9) never samples text, yet its screening EMR (0.956) sits inside the range of the generative API stacks, and the probability it assigns to the same option for the same abstract moved between identical calls for most abstracts. Non-determinism is therefore not a by-product of sampling that a different output format removes; it reaches a stack that returns distributions as well. What the distribution adds is a signal

[3] `https://platform.claude.com/docs/en/about-claude/model-deprecations`, accessed 9 October 2026.

the generative stacks do not provide: an abstract whose option probabilities move across runs is identifiable from the output itself; this study did not test whether routing such abstracts to a human reader improves screening.

Finding 8: Reproducible is not the same as usable. The three local stacks reach EMR=1.000 on extraction while returning schema-conforming output on 38–43% of calls (Section 4.10), and Mistral reproduces a degenerate screening rule perfectly (Finding 4). Reproducibility and usability are separable, cheap to measure together, and measured together in no study in Table 1.

Finding 9: A run card can record a configuration that was not executed. In the Claude execution at 1.0, the client attached the `temperature` field only when the requested value exceeded zero, so a request for 0 was sent without it and ran at the API default, while the run cards recorded `temperature: 0.0` because the provenance records were built from the requested configuration; the call hash, computed over the same requested values, could not detect the omission either. A single conditional, invisible in the configuration table, the run card and the hash, detached the executed configuration from the declared one. Provenance that records what was requested cannot detect this; provenance that records what was transmitted can.

5.2. Implications for Evidence Synthesis Practice

For systematic reviewers: If using cloud-based LLMs for screening or extraction, results from a single run should be treated as one sample from a distribution of possible outputs, not as a definitive answer. Running multiple repetitions and reporting variation metrics should become standard

practice, analogous to how human inter-rater reliability is routinely measured. It is tempting to place the stacks' screening EMR of 0.94–0.99 next to the $\kappa \geq 0.80$ target pre-specified for our human screeners and read the comparison as favourable, and we resist that reading for two reasons. First, the quantities are not commensurable: EMR and inter-run Fleiss' κ measure how consistently a *single* rater reproduces itself across repetitions, whereas Cohen's κ measures whether *two different* raters reach the same conclusion. A system can be perfectly self-consistent and consistently wrong (Finding 4). Second, on this same corpus our own two human raters reached $\kappa = 0.529$ (95% CI [0.383, 0.674]; Section 3.3), so the 0.80 figure functions here as an aspirational guideline rather than as a measured property of human screening. The honest comparison is therefore not "models versus the threshold" but "models and humans each measured, on different constructs, against a threshold that neither was shown to meet." Extraction EMR (0.15–0.47), by contrast, requires no such careful framing: it is below the 0.80 threshold under each of the constructs discussed above.

For journal editors and reviewers: Papers reporting LLM-assisted evidence synthesis should be expected to disclose: (a) the exact model version, (b) all inference parameters, (c) whether results were verified across multiple runs, and (d) provenance information enabling independent replication.

For tool developers: Open-weight models have already been scaled in this journal for regulatory information extraction in the hydropower domain;[1] our result adds a property to that choice which cost and control arguments do not cover, since the local single-device arm is the only config-

uration here that returned bit-identical output across all ten runs. The large difference in extraction EMR between local and cloud deployment suggests that deterministic local inference should be the default for reproducibility-critical applications. Local inference on consumer hardware (Apple M4, 24 GB RAM) cost $0.26 total for 174 hours across three models, versus $69.38 for 32 hours of API inference—a 266× advantage in total outlay. Normalised per hour of inference the gap is far wider still ($0.0015/h local vs. $2.14/h API, ≈1,400×; the local figure counts electricity only, not hardware, and the API figure excludes Gemini, which ran on the provider's free tier), since the local stacks ran five times longer for that outlay. Either way the cost advantage compounds with the reproducibility benefit; workload and cost scenarios for a review team are in Supplementary §25. Where cloud APIs must be used, every reported output should come with its run-to-run variation; whether majority voting across runs restores a stable output was not measured here.

5.3. Relationship to Prior Work

Table 1 compares this study with prior work.

Our screening flip rates (1.0% for Claude, 3.0% for GPT-4.1, 6.4% for Gemini) point in the same direction as the accuracy variations reported by Atil et al.[15] on general benchmarks; the two quantities differ (flip rate here, accuracy variation there), so the agreement is in direction only. Whole-output extraction changed far more often than the screening decision. The two EMRs are computed on different outputs and item sets, so the ratios below compare those outputs and do not measure how much the task itself amplifies non-determinism: Claude's non-match rate increased from 1.0% to

53.0% (53.0×; ≈88× if its two failed screening calls are not counted as non-matches), GPT-4.1's from 3.0% to 85.0% (28.3×), and Gemini's from 6.4% to 80.0% (12.5×).

The comparison with Jensen et al.[9] is particularly instructive. Their reported 94.1% inter-session reproducibility for ChatGPT-4o extraction appears to align with our API screening EMR (93.6–99.0%), creating the impression that API-based extraction is reasonably stable. However, their metric measures per-field agreement across 2 sessions, while our EMR measures whole-output identity across 10 runs. When we apply our stricter metric, extraction reproducibility is 15.0–47.0%. The two figures differ in metric, model, interface, corpus and number of runs, so their ratio is not attributable to the metric alone; our own field-level EMR (Supplementary Table 29) isolates the metric effect on the same runs. Per-field metrics miss the combinatorial explosion of variation: when 5 fields each have 97% individual reproducibility, the probability that *all* fields match simultaneously is only $0.97^5 = 0.859$—and this is before accounting for the variable-length effect estimate lists that drive most of our observed non-matches.

Gartlehner et al.'s[8] replication accuracies of 95.0–96.9% with Claude 2 were measured over two replications, each against human extraction, so they do not measure run-to-run agreement; a 10-run design with output hashing detects items that change in only 1–2 of 10 runs, which a 2-run comparison would usually miss.

The meta-analytic propagation experiment (Section 4.6) provides the most direct evidence of downstream impact. For Gemini, the number of usable effect estimates varied from 154 to 171 across runs—an 11.0% swing

in the evidence base feeding the pooled result. For GPT-4.1 the swing was 2.3% (176–180), and even the most stable API-served stack, Claude, showed a 1.0% swing (195–197). Across API-served stacks, 1–20% of articles produced a different number of effect estimates depending on the run, meaning that **the composition of a meta-analysis is not fixed when an LLM extracts the data**. Over the full corpus the pooled interval excluded RR=1 in every run; because the pooled set mixes increments, lags, outcomes and effect measures, this describes the propagation instrument at this k, not the $PM_{2.5}$–respiratory association. The per-run count analysis pools every extracted estimate, several per article, under a fixed-effect model that treats them as independent, so its intervals are too narrow; the stability statement rests on the random-effects re-analysis in the Supplement, which pools one estimate per article (k=39–63) and, under both DerSimonian–Laird and Hartung–Knapp–Sidik–Jonkman variances, changes no full-corpus conclusion. Run dependence appears at small k (Section 4.6).

5.4. Limitations

The results are bounded by the limitations below; for each we state what it prevents us from concluding.

Timing of the Claude stack. The Claude stack was run on 8 October 2026, eight months after the other five stacks, on a snapshot that Anthropic had announced for retirement (deprecated 30 September 2026, retirement 30 November 2026).[4] Provider-side changes to the serving stack in

[4] `https://platform.claude.com/docs/en/about-claude/model-deprecations`, accessed 9 October 2026.

that window cannot be excluded, so differences between Claude and the other API stacks cannot be attributed to the provider alone, and the temperature contrast of Finding 6 cannot be read as a pure temperature effect.

Stack sample. Six stacks—three local 7–9B models on one device, three proprietary API-served models—are not a sample of deployment practice. The local-versus-API contrast varies weights, infrastructure and serving layer together, and the one same-checkpoint comparison does not hold weight precision or the request payload constant. We therefore cannot attribute the gap to a specific serving component, nor claim that other hardware, other runtime versions or other API stacks behave the same way.

Ten runs per condition. With R=10, a per-item non-match probability of 0.30 or higher is detected with $\geq$95% power under a binomial model, while rarer instability (per-item $p<0.05$) is under-detected; the local EMR of 1.000 bounds the non-match rate at $\leq$0.6% (screening) and $\leq$3% (extraction) and does not exclude lower rates. EMR values here are not comparable with EMR computed over a different number of runs.

No within-stack seed ablation. Our data show that a transmitted seed does not buy determinism (Finding 6) and do not estimate how much the seed contributes on its own; that estimate needs one stack run with and without the seed, which we did not execute.

An agreement coefficient below its target. Two blinded raters working from a pre-registered protocol reached κ=0.529 (95% CI [0.383, 0.674]) against a target of 0.80, and the shortfall is statistically decisive ($z = -3.65$). The disagreement is located: it is directional (exact McNemar $p=2.2\times10^{-4}$), and 17 of the 19 asymmetric discordances turn on a single inclusion criterion

whose original wording admitted two defensible readings; showing that the amended criterion repairs it requires a fresh independent sample. This prevents us from interpreting sensitivity or certifying the reference standard; it does not affect EMR, which needs no reference. Because the reference excludes were sampled from clearly off-topic records, the specificities in Table 3 do not show how often a stack would include a near-miss study.

Usability of the local outputs. The three local stacks reach EMR=1.000 on extraction while returning schema-conforming output on 38–43% of calls. This paper therefore does not show that local stacks yield usable extractions.

Provenance of the transmitted payload. The released harness hashes the declared configuration and records a transmitted value only for the Claude stack's temperature, so a field dropped from any other stack's request body would not be detected by the deposited records. In the same-checkpoint cloud arm, the per-call provider label reads `groq`, a fixed string left in its script; the run cards record the route actually used (OpenRouter to DeepInfra).

Unlogged retries. Because unparseable first responses were re-requested and attempts were not logged, we cannot state how much this resampling raised parse success or lowered EMR for any stack.

No structured-output modes. Every stack received prompt-level JSON instructions through its standard endpoint (Table 2); none used the providers' structured-output or tool-use modes, so we cannot say whether those modes change extraction EMR or schema conformance.

Screening EMR covers the decision only. It ignores confidence scores and rationales, so it does not measure their variation; a full-output

screening EMR could only be equal or lower.

Corpus composition. Sixty per cent of the corpus comes from the rule's low-confidence stratum. It would be tempting to read these flip rates as upper bounds for standard workloads, on the reasoning that a typical review contains fewer borderline cases. That reading does not survive inspection of the stratum: 260 of its 300 abstracts carry the maximum inclusion score of 5, the same score borne by every clear-include, and all 300 carry zero exclusion reasons. The stratum marks where the automated rule was unconfident, which is not the same as where a human reader would hesitate, and its composition sits closer to the include side than to a genuine borderline. We therefore make no directional claim about how these rates transfer to other corpora. Whether a review with fewer such abstracts would flip less often is untested here, and the enrichment fraction is a property of our sampling frame rather than a calibrated difficulty setting.

Snapshot validity. For the three API-served stacks, the infrastructure and intermediate-precision routines—GPU topology, batching policy, weight checkpoint version, internal post-training—are opaque to the client and may change silently. The main stacks ran within a 3-week window (February 11–March 2, 2026), the same-checkpoint cloud stack and the Gemini and GPT-4.1 fixed-slot runs in April 2026, and the Claude and Jev arms on 8 October 2026. Our API results describe the snapshots observed in those windows and cannot be extrapolated to later provider versions.

Single corpus and domain. All measurements come from one 500-abstract corpus on $PM_{2.5}$ and respiratory hospitalization, a literature that reports effects in a small set of formats (RR or OR with 95% CI, per 10 $\mu g/m^3$

or per interquartile range). We therefore cannot say whether the extraction EMR values or the screening-to-extraction contrast hold for less standardized environmental literatures, for other periods, or for other domains.

Single local device. The local EMR=1.000 was obtained on one device (Apple M4, 24 GB RAM, macOS Darwin 24.6.0, Ollama v0.15.5; the Ollama version is taken from the experiment log, not from the run cards). We cannot claim that the same local stacks are deterministic on other hardware or runtime versions, nor that they reproduce these exact outputs there.

Pre-registration disclosure. This study is partly pre-registered, and we state precisely which parts, because the distinction matters for how much weight each result can carry. The core analyses (Sections 4.4–4.6) are exploratory: the choice of 10 repetitions, whole-output hash EMR, four semantic-tier definitions, and fixed-effect meta-analytic propagation were made during analysis design but without an Open Science Framework or AsPredicted submission. The dual-human labeling validation is the exception and is *formally* pre-registered: the protocol, the screening templates, and the agreement script were frozen as an OSF registration on 2026-05-12 (`https://doi.org/10.17605/OSF.IO/FGN3E`), two months before either label set was returned, and that registration pre-specified the $\kappa \geq 0.80$ target together with the contingency followed in Section 3.3 when the target was missed. The fixed-slot, same-checkpoint and silver-external analyses, the Claude stack at temperature=0 and the Jev arm are likewise not pre-registered. Their results therefore describe this dataset and cannot be read as confirmatory tests of hypotheses fixed before data collection.

5.5. Recommendations

Practical strategies for reproducible geoscientific data analysis, which already list non-deterministic generative-AI output among the barriers, have been set out in this journal.[3] The recommendations below are specific to LLM-mediated steps and add to that practice rather than replace it.

1. **Run each LLM step repeatedly** and report EMR together with the number of runs, since EMR declines as runs are added (Section 4.7); ten runs detect a per-item non-match probability of 0.30 with $\geq$95% power, and fewer runs detect less (Section 5.4).
2. **Record the transmitted request payload** and its hash in the call record, not the requested configuration.
3. **Report the deployment stack**—weights, provider, infrastructure, API layer and access dates—not only the model name.
4. **Do not treat temperature zero or a seed as a reproducibility control**; verify by repetition.
5. **Report schema conformance and accuracy alongside EMR.**
6. **Prefer local single-device serving** for reproducibility-critical steps when capability permits, with hardware and software versions pinned.
7. **Where a model returns probabilities**, report how far the probabilities move across runs.

6. Conclusion

LLM steps entering environmental evidence pipelines do not inherit the re-execution guarantee that the rest of the modelling toolchain provides.

Across 35,636 calls, local single-device stacks reproduced their outputs exactly, while API-served stacks at temperature zero returned an identical complete extraction for 15–47% of articles. A model that reproduced exactly on a local stack varied across runs when served in the cloud, so a model name does not specify the experimental condition. A transmitted seed, a fixed-slot output shape and a decision model returning probabilities each left the variation in place; temperature zero did not remove it, and one provider now rejects non-default temperatures on its newer models. Reproducibility was also separate from usability: the perfectly reproducible local stacks returned schema-conforming extractions on 38–43% of calls.

Frameworks that assess computational reproducibility process by process[39] need a per-process check for LLM-mediated steps. Whole-output EMR and schema conformance provide it, provided the call records store the transmitted payload rather than the requested one (Finding 9). Because per-vendor figures expire with each provider update, re-running the audit on a new provider snapshot is one command per stack.

Acknowledgements

The authors thank the two independent raters who labeled the validation subset **[PENDING: names, or removal of this sentence if both raters are listed as co-authors]**.

Funding Statement

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors. API costs for the Claude, Gem-

ini, GPT-4.1 and Jev experiments were funded by the authors.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude (Anthropic) for code development assistance and language editing. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. All numerical results, tables and figures are computed by the deposited scripts from the deposited raw outputs.

Data Availability Statement

All code, data, prompts, schemas, raw outputs, and run provenance records are publicly available at `https://github.com/Roverlucas/llm-evidence-synthesis-re` and archived at the Open Science Framework (`https://doi.org/10.17605/OSF.IO/VR934`). The dual-human labeling protocol is deposited separately as a frozen OSF registration (`https://doi.org/10.17605/OSF.IO/FGN3E`, 2026-05-12), which pre-dates label collection; the returned labels, agreement statistics, and the v1.2 protocol amendment are in the repository under `data/dual_labeling/` ([**PENDING: repository tag for the submitted version**]). API keys are excluded from the repository. The 500 PubMed abstract identifiers, rule and human labels, and complete analysis scripts are included to enable full reproduction of the reported results.

CRediT authorship contribution statement

Lucas Rover (L.R.) conceived the study, designed the protocol, developed the software, conducted all experiments and analyses, and wrote the manuscript. **Yara de Souza Tadano** (Y.d.S.T.) supervised the research, contributed to methodology design, and reviewed the manuscript.

References

[1] H.-J. Yoon, F. B. Ashraf, T. A. Ruggles, and D. Singh. "Scaling open-weight large language models for hydropower regulatory information extraction: A systematic analysis". In: *Environmental Modelling & Software* 203 (2026), p. 107030. DOI: 10.1016/j.envsoft.2026.107030.

[2] E. Park, J. Park, T. Kim, and A. Jin. "What structured tool interfaces do and do not provide for LLM-based SWAT+ calibration". In: *Environmental Modelling & Software* 205 (2026), p. 107142. DOI: 10.1016/j.envsoft.2026.107142.

[3] M. Schlögl, L. Waltersdorfer, P. Regner, A. Siposova, and A. Brenning. "Overcoming barriers to reproducibility in geoscientific data analysis: Challenges and practical implementation strategies". In: *Environmental Modelling & Software* 200 (2026), p. 106962. DOI: 10.1016/j.envsoft.2026.106962.

[4] J. P. T. Higgins, J. Thomas, J. Chandler, et al. *Cochrane Handbook for Systematic Reviews of Interventions*. 2nd. Wiley, 2019.

[5] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al. “The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews”. In: *BMJ* 372 (2021), n71.

[6] R. Borah, A. W. Brown, P. L. Capers, and K. A. Kaiser. “Analysis of the Time and Workers Needed to Conduct Systematic Reviews”. In: *BMJ Open* 7.4 (2017), e012545.

[7] T. Oami, Y. Okada, and T.-a. Nakada. “Performance of a Large Language Model in Screening Citations”. In: *JAMA Network Open* 7.7 (2024). PMID: 38976267. Published 1 July 2024, e2420496.

[8] G. Gartlehner, L. Kahwati, R. Hilscher, et al. “Data Extraction for Evidence Synthesis Using a Large Language Model: A Proof-of-Concept Study”. In: *Research Synthesis Methods* 15.4 (2024), pp. 576–589.

[9] M. M. Jensen, M. B. Danielsen, J. Riis, K. A. Kristjansen, S. Andersen, Y. Okubo, et al. “ChatGPT-4o Can Serve as the Second Rater for Data Extraction in Systematic Reviews”. In: *PLOS ONE* 20.1 (2025), e0313401. DOI: 10.1371/journal.pone.0313401.

[10] Y. Li, S. Datta, M. Rastegar-Mojarad, K. Lee, H. Paek, J. Glasgow, C. Liston, L. He, X. Wang, and Y. Xu. “Enhancing systematic literature reviews with generative artificial intelligence: development, applications, and performance evaluation”. In: *Journal of the American Medical Informatics Association* 32.4 (2025), pp. 616–625. DOI: 10.1093/jamia/ocaf030.

[11] D. Scherbakov et al. “The Emergence of Large Language Models as Tools in Literature Reviews”. In: *Journal of the American Medical Informatics Association* 32.5 (2025), ocaf063.

[12] K. Matsui, T. Utsumi, Y. Aoki, T. Maruki, M. Takeshima, and Y. Takaesu. “Human-Comparable Sensitivity of Large Language Models in Identifying Eligible Studies Through Title and Abstract Screening”. In: *Journal of Medical Internet Research* 26 (2024), e52758.

[13] T. Oami, Y. Okada, and T.-a. Nakada. “Optimal Large Language Models to Screen Citations for Systematic Reviews”. In: *Research Synthesis Methods* 16.6 (2025). doi:10.1017/rsm.2025.10014, pp. 859–875.

[14] O. Farotimi, A. Dunn, C. J. Van Lissa, J. R. Polanin, D. Mavridis, and T. D. Pigott. “Guidance for manuscript submissions testing the use of generative AI for systematic review and meta-analysis”. In: *Research Synthesis Methods* 17.2 (2025). Editorial guidance on reporting standards for GenAI-assisted evidence synthesis; published 11 December 2025, pp. 237–239. DOI: 10.1017/rsm.2025.10058.

[15] B. Atil, S. Aykent, A. Chittams, et al. “Non-Determinism of “Deterministic” LLM Settings”. In: *arXiv preprint* (2024). arXiv:2408.04667.

[16] E. Klishevich, Y. Denisov-Blanch, S. Obstbaum, I. Ciobanu, and M. Kosinski. “Measuring Determinism in Large Language Models for Software Code Review”. In: *arXiv preprint* (2025). arXiv:2502.20747.

[17] T. Fu, G. Martínez, J. Conde, C. Arriaga, P. Reviriego, X. Qi, and S. Liu. “Beyond Reproducibility: Token Probabilities Expose Large Language Model Nondeterminism”. In: *arXiv preprint* (2026). arXiv:2601.06118.

[18] G. Karelin, K. Nakajima, E. F. Soto-Astorga, E. Carr, M. James, and T. Froese. “Indeterminism in Large Language Models: An Unintentional Step Toward Open-Ended Intelligence”. In: *Preprint submitted to Artificial Life* (2025). PhilSci-Archive 26807; preprint announced September 2025. URL: https://philsci-archive.pitt.edu/26807/.

[19] M. Hutson. “Artificial Intelligence Faces Reproducibility Crisis”. In: *Science* 359.6377 (2018), pp. 725–726.

[20] B. Haibe-Kains, G. A. Adam, A. Hosny, et al. “Transparency and Reproducibility in Artificial Intelligence”. In: *Nature* 586 (2020), pp. 14–16.

[21] S. Kapoor and A. Narayanan. “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science”. In: *Patterns* 4.9 (2023), p. 100804.

[22] H. Han. “Challenges of Reproducible AI in Biomedical Data Science”. In: *BMC Medical Genomics* 18.Suppl 1 (2025), p. 8. DOI: 10.1186/s12920-024-02072-6.

[23] A. Desai, M. Abdelhamid, and N. R. Padalkar. “What Is Reproducibility in Artificial Intelligence and Machine Learning Research?” In: *AI Magazine* 46.2 (2025), e70004. DOI: 10.1002/aaai.70004.

[24] H. Semmelrock, T. Ross-Hellauer, S. Kopeinik, D. Theiler, A. Haberl, S. Thalmann, and D. Kowald. “Reproducibility in Machine-Learning-Based Research: Overview, Barriers, and Drivers”. In: *AI Magazine* 46.1 (2025), e70002.

[25] Q. Khraisha et al. “Can large language models replace humans in systematic reviews? Evaluating GPT-4’s efficacy in screening and extracting data from peer-reviewed and grey literature in multiple languages”. In: *Research Synthesis Methods* 15.4 (2024), pp. 616–626. DOI: 10.1002/jrsm.1715.

[26] Z. Wang, L. Cao, B. Danek, Q. Jin, Z. Lu, and J. Sun. “Accelerating Clinical Evidence Synthesis with Large Language Models”. In: *npj Digital Medicine* 8 (2025), p. 509. DOI: 10.1038/s41746-025-01840-7.

[27] C. Cao et al. “Automation of Systematic Reviews with Large Language Models”. In: *medRxiv* (2025). Preprint. doi:10.1101/2025.06.13.25329541.

[28] M. A. Khan et al. “Collaborative Large Language Models for Automated Data Extraction in Living Systematic Reviews”. In: *Journal of the American Medical Informatics Association* 32.4 (2025). doi:10.1093/jamia/ocae325, pp. 638–647.

[29] L. Li, A. Mathrani, and T. Susnjak. “Transforming Evidence Synthesis: A Systematic Review of the Evolution of Automated Meta-Analysis in the Age of AI”. In: *Research Synthesis Methods* 17.3 (2026), pp. 403–450. DOI: 10.1017/rsm.2025.10065.

[30] F. Angermeir, M. Amougou, M. Kreitz, A. Bauer, M. Linhuber, D. Fucci, F. Moyón C., D. Mendez, and T. Gorschek. “Reflections on the Reproducibility of Commercial LLM Performance in Empirical Software Engineering Studies”. In: arXiv:2510.25506. 2025.

[31] B. Yu. “After Computational Reproducibility: Scientific Reproducibility and Trustworthy AI”. In: *Harvard Data Science Review* 6.1 (2024).

[32] S. Longpre, R. Mahari, A. Chen, et al. “A Large-Scale Audit of Dataset Licensing and Attribution in AI”. In: *Nature Machine Intelligence* 6.8 (2024), pp. 975–987. DOI: 10.1038/s42256-024-00878-8.

[33] W. Liang, N. Rajani, X. Yang, E. Ozoani, E. Wu, Y. Chen, et al. “Systematic Analysis of 32,111 AI Model Cards Characterizes Documentation Practice”. In: *Nature Machine Intelligence* 6.7 (2024), pp. 744–753. DOI: 10.1038/s42256-024-00857-z.

[34] R. Bommasani, K. Klyman, S. Longpre, et al. “The Foundation Model Transparency Index v1.1”. In: *Transactions on Machine Learning Research* (2024).

[35] S. Leo, M. R. Crusoe, L. Rodríguez-Navas, et al. “Recording Provenance of Workflow Runs with RO-Crate”. In: *PLOS ONE* 19.9 (2024), e0309210.

[36] A. Kale, T. Nguyen, F. C. Harris, C. Li, J. Zhang, and X. Ma. “Provenance Documentation to Enable Explainable and Trustworthy AI: A Literature Review”. In: *Data Intelligence* 5.1 (2023), pp. 139–162.

[37] S. Longpre et al. “Data Authenticity, Consent, and Provenance for AI Are All Broken”. In: *Proceedings of ICML 2024 (Position Paper)*. arXiv:2404.12691. 2024.

[38] L. Rover, H. V. Siqueira, E. T. Bacalhau, A. T. de Azevedo, and Y. d. S. Tadano. “Hidden Non-Determinism in LLM APIs Undermines Scientific Reproducibility”. In: *Nature Communications* (2026). Accepted.

[39] Z. Zhu, M. Chen, G. Ren, Y. He, L. Sun, F. Zhang, Y. Wen, S. Yue, and G. Lü. "A framework for assessing the computational reproducibility of geo-simulation experiments". In: *Environmental Modelling & Software* 186 (2025), p. 106323. DOI: 10.1016/j.envsoft.2025.106323.

[40] W. Yu, R. Xu, T. Ye, et al. "Estimates of Global Mortality Burden Associated with Short-Term $PM_{2.5}$ Exposure". In: *The Lancet Planetary Health* 8.2 (2024), e146–e155.

[41] Y. d. S. Tadano, E. T. Bacalhau, L. Casacio, E. Puchta, T. S. Pereira, T. A. Alves, C. M. L. Ugaya, and H. V. Siqueira. "Unorganized Machines to Estimate the Number of Hospital Admissions Due to Respiratory Diseases Caused by PM_{10} Concentration". In: *Atmosphere* 12.10 (2021). doi:10.3390/atmos12101345, p. 1345. DOI: 10.3390/atmos12101345.

[42] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, et al. "The LLaMA 3 Herd of Models". In: *arXiv preprint* (2024). arXiv:2407.21783.

[43] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, et al. "Mistral 7B". In: *arXiv preprint* (2023). arXiv:2310.06825.

[44] Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, et al. "Gemma 2: Improving Open Language Models at a Practical Size". In: *arXiv preprint* (2024). arXiv:2408.00118.

[45] Anthropic. *Claude Sonnet 4.5 System Card*. Tech. rep. https://docs.anthropic.com/en/docs/about-claude/models. Anthropic, 2025.

[46] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, et al. “Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Agentic Capabilities”. In: *arXiv preprint* (2025). arXiv:2507.06261.

[47] OpenAI. *GPT-4.1: A New Generation of OpenAI Models*. Tech. rep. Released April 2025. `https://openai.com/index/gpt-4-1/`. OpenAI, 2025.

[48] J. A. Hanley and A. Lippman-Hand. “If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators”. In: *Journal of the American Medical Association* 249.13 (1983), pp. 1743–1745.